

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | [LinkedIn Profile](#)



6h00 pm – 9h50 pm

- Monday A -G207
- Tuesday B-G204
- Wednesday E-106
- Thursday F-G307

April 2025

Database development with PL/SQL



Reference reading

- [Difference between Cursor and Trigger in DBMS](#)
- [Difference between Cursor and Trigger in PLSQL](#)
- [9 Using Triggers](#)
- [CREATE TRIGGER](#)
- [Oracle PL/SQL Trigger: A Comprehensive Guide](#)
- [Auditing employee table changes with PL/SQL Trigger](#)
- [Oracle PL/SQL](#)

Lecture 08 – PLSQL Triggers and Audit Mechanisms

The objectives of triggers in PL/SQL

What is a Trigger in PL/SQL?

A **trigger** in PL/SQL is a **predefined program or block of code** that is automatically executed (or "fired") when a specific event occurs in the database. It is stored in the database and is triggered by actions such as **INSERT, UPDATE, DELETE, or DDL operations** on a table, view, schema, or database.

Key Features of Triggers:

- **Automatic Execution:** Triggers execute automatically when the specified event occurs.
- **Data Integrity:** They help enforce business rules and maintain consistency in the database.
- **Error Handling:** Triggers can validate data before changes are made.
- **Event-driven:** They respond to specific actions like **INSERT, UPDATE, DELETE**, or system events like **LOGON, SHUTDOWN**.

Types of Triggers in PL/SQL:

- **DML Triggers** – Fired on **INSERT, UPDATE, DELETE** operations.
- **DDL Triggers** – Fired on **CREATE, ALTER, DROP** statements.
- **Database Triggers** – Fired on system events like **LOGON, LOGOFF, STARTUP, SHUTDOWN**.
- **INSTEAD OF Triggers** – Used for views that do not allow direct modifications.

When to Use Triggers?

- **Auditing changes** (e.g., tracking salary modifications, order status updates).
- **Enforcing business rules** (e.g., preventing unauthorized deletions).
- **Automatically updating related tables** (e.g., updating stock levels when an order is placed).

Oracle – PLSQL Database Triggers

1. In Oracle PL/SQL, a **trigger** is a named block of PL/SQL code that is **automatically executed (or "fired")** in response to specific events on a particular table, view, schema, or database. **Triggers can enforce business rules, maintain audit trails, or perform automated actions.**
2. A **Trigger** is a PL/SQL block that **executes implicitly** in response to a specific database event (e.g., INSERT, UPDATE, DELETE) on a table or view. It is planned for a particular **event** (such as data changes) and **timing** (e.g., BEFORE or AFTER the event).

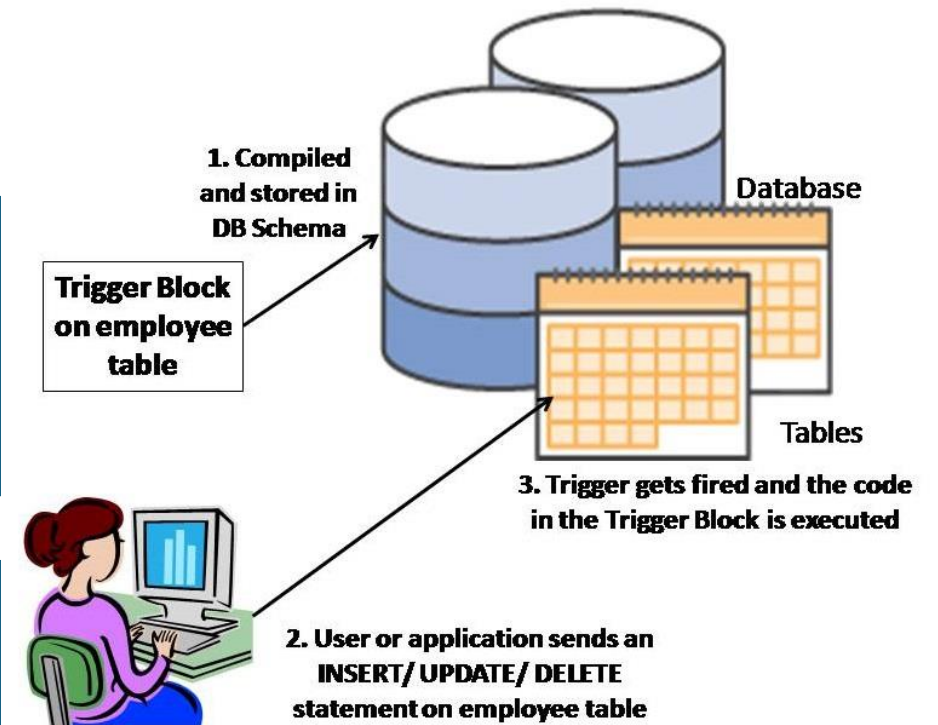


Characteristics:

- **Executed Implicitly:** Trigger fires automatically when the specified event occurs.
- **Event-Driven:** Associated with DML operations (e.g., INSERT, UPDATE, DELETE).
- **Centralized Actions:** Performs centralized and related actions to enforce data integrity, auditing, or automation.
- **Complexity:** Excessive use of triggers can lead to complex interdependencies between database objects.

Real-world Example of Trigger Usage:

- **Auditing:** Automatically log details of changes (e.g., who modified data, when, and what changed) into a separate audit table.
- **Data Validation:** Ensure specific business rules (e.g., salary cannot exceed a certain threshold) are enforced before inserting or updating a row.
- **Cascading Updates:** Update related tables to maintain data integrity when changes are made.



Source Image: <https://csveda.com/pl-sql-trigger-basics-uses/>

Triggers are a powerful feature in relational databases like Oracle, MySQL, or PostgreSQL, allowing automation and ensuring consistency within the database. However, overuse can lead to performance overhead, so they should be employed carefully.



PL/SQL Triggers – Advantages

Automation: Automates complex calculations and validations.

Data Integrity: Maintains consistency between tables (e.g., referential integrity).

Logging and Auditing: Tracks changes for insights into data usage patterns.

Real-time Synchronization: Ensures data consistency across multiple tables.

Security Enforcement: Prevents invalid transactions and restricts certain actions.

PROS

Syntax:

```
CREATE [ OR REPLACE ] TRIGGER <trigger_name>
```

```
[ BEFORE | AFTER | INSTEAD OF ]
```

Trigger Timing

```
[ INSERT | UPDATE | DELETE.....]
```

Event

```
ON <name of underlying object>
```

```
[ FOR EACH ROW ]
```

Row Level

```
[ WHEN <condition for trigger to get execute> ]
```

Conditional Clause

```
DECLARE
```

```
<Declaration part>
```

```
BEGIN
```

```
<Execution part>
```

```
EXCEPTION
```

```
<Exception handling part>
```

```
END;
```

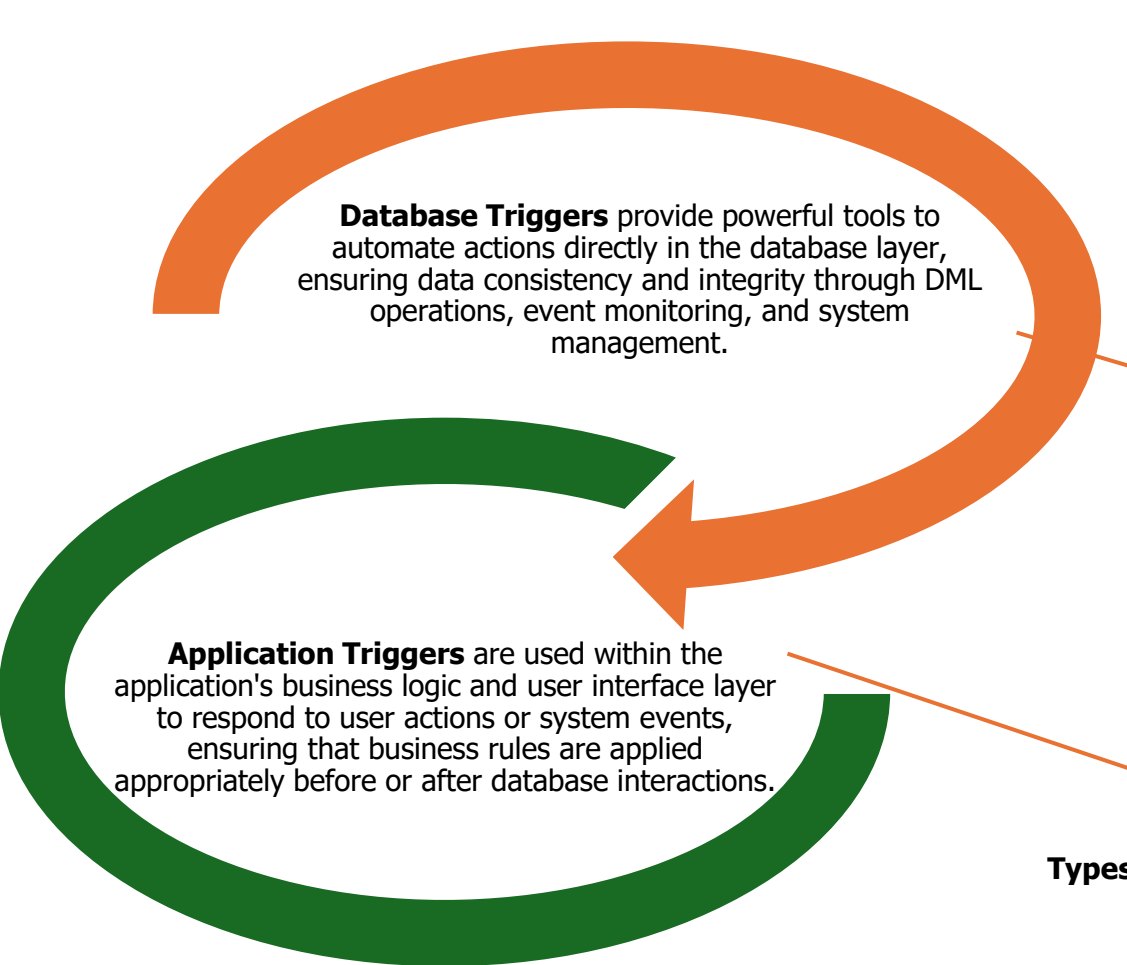
Source Image: <https://www.guru99.com/triggers-pl-sql.html>

PL/SQL Triggers – Disadvantages



Disadvantage	Description	Impact
Performance Overhead	Triggers add processing time to INSERT, UPDATE, or DELETE operations.	Can slow down database performance, especially for high-traffic tables or large datasets.
Debugging and Maintenance	Triggers execute implicitly, making them hard to debug and trace.	Difficult for developers to track issues or understand the flow of data modifications.
Hidden Logic	Business logic in triggers is not immediately visible in application code.	Causes confusion and reduces transparency, leading to unexpected behavior.
Increased Complexity	Triggers can cause unintended cascading effects (e.g., one trigger firing another).	Creates complex dependencies, making the database harder to manage.
Limited Use Cases	Triggers are bound to database events and cannot handle all types of application logic.	Requires additional methods (e.g., stored procedures or application logic) for more complex scenarios.
Recursive Execution	Poorly designed triggers may invoke themselves or other triggers recursively.	Can lead to infinite loops, database crashes, or data corruption.
Portability Issues	Trigger syntax and behavior vary across database platforms.	Challenges arise when migrating to a different database system.
Difficulty in Testing	Triggers execute in the background, making them hard to test independently.	Ensuring expected behavior in all scenarios requires extensive testing.
Over-reliance on Triggers	Developers may use triggers for tasks better handled at the application layer.	Blurs the separation between application and database logic, complicating system maintenance.

Triggers Classification: Application & Database Trigger



Database Triggers provide powerful tools to automate actions directly in the database layer, ensuring data consistency and integrity through DML operations, event monitoring, and system management.

Application Triggers are used within the application's business logic and user interface layer to respond to user actions or system events, ensuring that business rules are applied appropriately before or after database interactions.

```
CREATE OR REPLACE TRIGGER check_salary
-- The trigger fires before an INSERT operation occurs on the 'employees' table
BEFORE INSERT ON employees
-- The trigger executes for each row affected by the INSERT statement
FOR EACH ROW
BEGIN
-- Check if the new salary being inserted is less than 1000
IF :NEW.salary < 1000 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Salary must be above 1000');
END IF;
END;
```



```
// Java pseudo code example to demonstrate application trigger
public void onSalaryInsert(Employee emp) {
    if (emp.getSalary() < 1000) {
        throw new IllegalArgumentException("Salary must be greater than or
        equal to 1000");
    }
    // Code to insert employee to the database
}
```

Types:

- **UI Event Handlers:** Trigger actions when a user interacts with the interface (e.g., button clicks).
- **Business Logic Triggers:** Trigger specific actions or calculations based on certain conditions.
- **Database Interactions:** Trigger actions after database operations (e.g., after a record is updated).
- **Use Cases:** Business rule enforcement, user interaction handling, database operation responses.

Trigger Syntax, Behavior of Database Triggers



```
-- Creates or replaces a trigger to execute logic before/after specific table events.  
CREATE [OR REPLACE] TRIGGER trigger_name
```

```
-- Determines trigger timing relative to the event (BEFORE, AFTER, or INSTEAD OF).  
{BEFORE | AFTER | INSTEAD OF}
```

```
-- Specifies the triggering events (INSERT, UPDATE, DELETE).  
{INSERT [OR] | UPDATE [OR] | DELETE}
```

```
-- Monitors specific columns for UPDATE events (optional).  
[OF col_name]
```

```
-- Defines the table or view the trigger applies to.  
ON table_name
```

```
-- References old and new row values for processing (optional).  
[REFERENCING OLD AS o NEW AS n]
```

```
-- Executes the trigger for each affected row (optional).  
[FOR EACH ROW]
```

```
-- Executes the trigger conditionally if the specified condition is true.  
WHEN (condition)
```

```
BEGIN
```

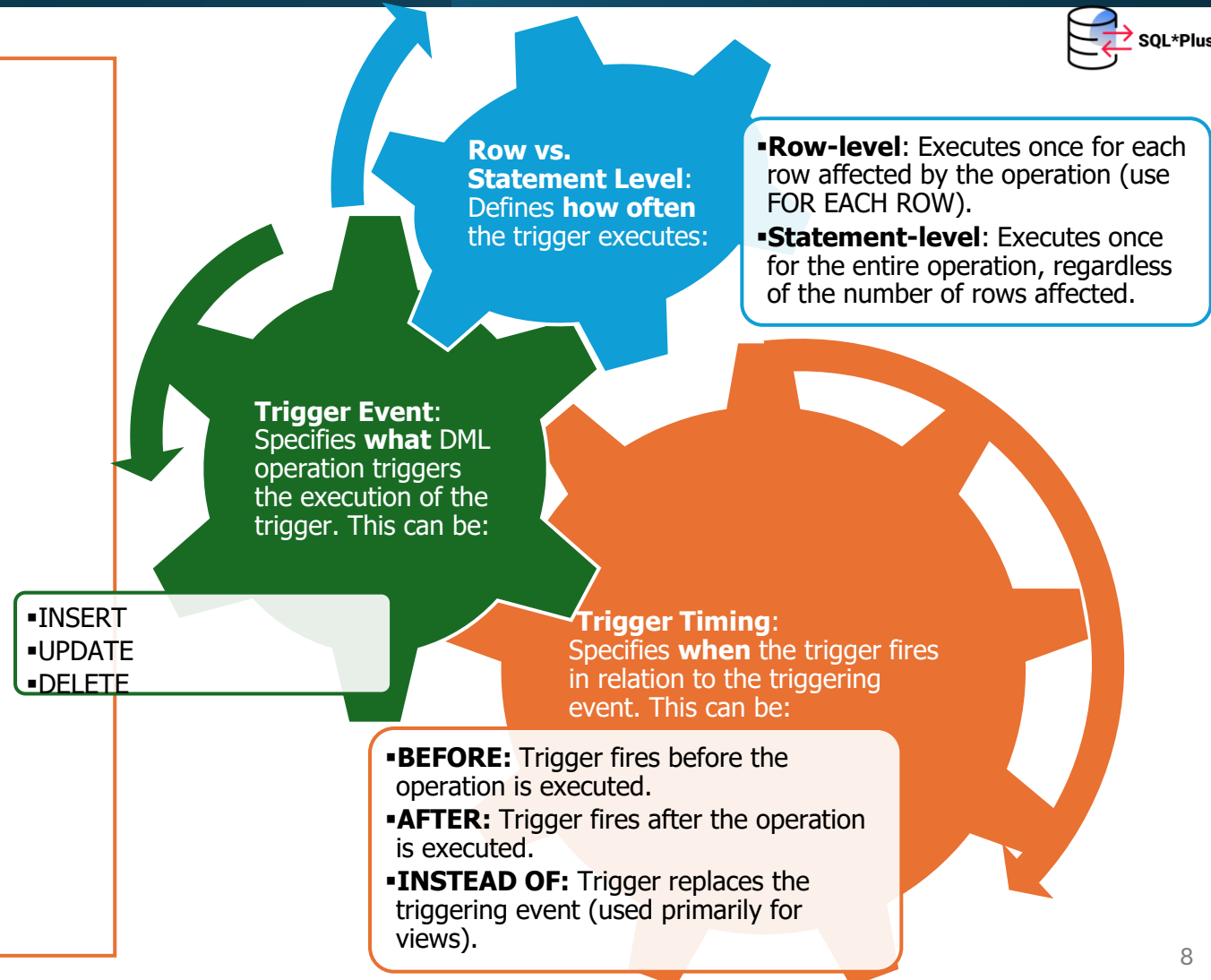
```
-- Define the logic executed when the trigger fires.  
NULL; -- Replace with actual trigger logic.
```

```
EXCEPTION
```

```
-- Define logic for handling errors during trigger execution.  
NULL; -- Replace with actual error handling logic
```

```
END;
```

```
/
```



Where,

CREATE [OR REPLACE] TRIGGER trigger_name – Creates or replaces an existing trigger with the *trigger_name*.

{BEFORE | AFTER | INSTEAD OF} – This specifies when the trigger will be executed. The INSTEAD OF clause is used for creating trigger on a view.

{INSERT [OR] | UPDATE [OR] | DELETE} – This specifies the DML operation.

[OF col_name] – This specifies the column name that will be updated.

[ON table_name] – This specifies the name of the table associated with the trigger.

[REFERENCING OLD AS o NEW AS n] – This allows you to refer new and old values for various DML statements, such as INSERT, UPDATE, and DELETE.

[FOR EACH ROW] – This specifies a row-level trigger, i.e., the trigger will be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.

WHEN (condition) – This provides a condition for rows for which the trigger would fire. This clause is valid only for row-level triggers.

When to Use Triggers

1. DML Triggers:

Use **DML Triggers** to automate actions in response to changes in data. These are ideal for:

- Logging changes to tables (e.g., tracking updates or deletions).
- Enforcing business rules and integrity constraints automatically when INSERT, UPDATE, or DELETE operations occur.

2. INSTEAD OF Triggers:

Use **INSTEAD OF Triggers** to enable INSERT, UPDATE, or DELETE operations on views that are otherwise non-updatable. These triggers allow you to define custom logic for handling DML operations on such views.

3. DDL Triggers:

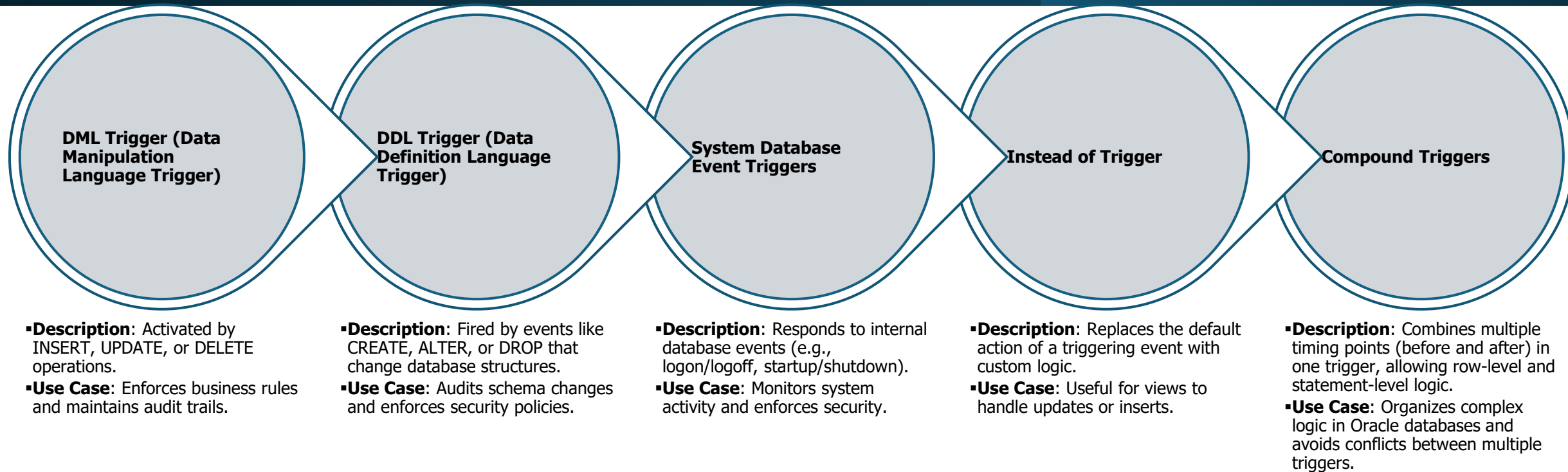
Use **DDL Triggers** to monitor or control schema-level changes. These triggers are helpful for:

- Auditing schema modifications (e.g., DROP, ALTER, or CREATE operations).
- Restricting unauthorized schema changes at the database or schema level.

Caution with Triggers

- Avoid excessive use of triggers to prevent **complex interdependencies**.
- Excessive use can degrade performance and make debugging difficult.
- Disable or drop triggers temporarily when troubleshooting or performing bulk operations.

Types of Database Triggers



Summary

These triggers help automate actions, enforce rules, and maintain data integrity in database operations.



Row-Level Trigger

Row-Level Trigger in SQL

A **row-level trigger** is a type of trigger that is fired for each row affected by a Data Manipulation Language (DML) operation such as INSERT, UPDATE, or DELETE. It executes **once for each row** that is modified, and it allows you to perform actions based on the data in the individual rows before or after the operation.

Row-level triggers are defined using the FOR EACH ROW clause, and they have access to the **:NEW** and **:OLD** pseudorecords, which store the values of the columns in the affected rows.

Structure of a Row-Level Trigger

```
CREATE OR REPLACE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
FOR EACH ROW
DECLARE
  -- Declare any local variables if needed
BEGIN
  -- Trigger logic goes here
  -- You can access :NEW and :OLD for the values of columns
END;
```

- **BEFORE:** Executes before the DML operation (INSERT, UPDATE, or DELETE).
- **AFTER:** Executes after the DML operation.
- **INSERT, UPDATE, DELETE:** Specifies the type of DML operation that will fire the trigger.
- **:NEW:** Refers to the new value in the row during INSERT or UPDATE. For DELETE, this is NULL.
- **:OLD:** Refers to the old value in the row during UPDATE or DELETE. For INSERT, this is NULL.

BEFORE vs AFTER: What's the difference in this context?

Aspect	BEFORE Trigger	AFTER Trigger
When it fires	Just before the row is updated	Just after the row is updated
Can access :OLD?	✅ Yes	✅ Yes
Can access :NEW?	✅ Yes and can modify it	✅ Yes (read-only; cannot modify :NEW)
Use case	Validation, transformation, custom logic	Auditing, logging, sending notifications

Example of a Row-Level Trigger: Logging Salary Changes

In this example, we will create a row-level trigger that logs every salary change to an audit table.

Trigger and Audit Table Definition

```
-- Create an audit table to store salary changes
CREATE TABLE salary_audit (
  employee_id NUMBER,
  old_salary NUMBER,
  new_salary NUMBER,
  change_date DATE
);

-- Create the row-level trigger
CREATE OR REPLACE TRIGGER log_salary_changes
AFTER UPDATE ON employees
FOR EACH ROW
WHEN (OLD.salary != NEW.salary)
BEGIN
  -- Insert a record into the salary_audit table whenever salary is updated
  INSERT INTO salary_audit (employee_id, old_salary, new_salary, change_date)
  VALUES (:OLD.employee_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
```

Explanation:

- **Trigger Event:** AFTER UPDATE -This trigger is fired **after** an UPDATE operation on the employees table.
- **Row-Level:** FOR EACH ROW -This trigger fires once for each row affected by the UPDATE operation.
- **Audit Action:** Inside the trigger body, it logs the employee's ID, old salary, new salary, and the date of the change into the salary_audit table using the INSERT statement.

Behavior:

- **After Update:** After each salary change, the trigger inserts a record into the salary_audit table. This allows you to keep track of salary changes over time.

Summary

- **Row-level triggers** execute once for each affected row in a DML operation.
- **FOR EACH ROW** is used to specify row-level behavior.
- The **:NEW** and **:OLD** pseudorecords allow you to reference the new and old values of the columns being modified.
- They are useful for tasks like validation, logging, or enforcing business rules on a row-by-row basis.

By using row-level triggers effectively, you can ensure that your database enforces business rules and data integrity in a transparent and automated way. However, it's important to balance their use with performance considerations.

Row-Level Trigger

What This Trigger Does

- **Event:** *BEFORE UPDATE OF SALARY* – fires before a salary update on the *EMPLOYEES* table.
- **Level:** *FOR EACH ROW* – row-level trigger (runs once per updated row).
- **Filter:** *WHEN (NEW.ROLE_ID <> 1)* – skips employees with *ROLE_ID = 1*, e.g., "President".
- **Logic:**
 - Computes salary difference.
 - Logs output to the console (via *DBMS_OUTPUT*), which is helpful for debugging or audit-like feedback during development.

```
-- Step 1: Enable SERVEROUTPUT
SET SERVEROUTPUT ON SIZE UNLIMITED;

CREATE OR REPLACE TRIGGER log_salary_changes
BEFORE UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW
WHEN (NEW.ROLE_ID <> 1)
DECLARE
    sal_diff NUMBER;
BEGIN
    -- Calculate the salary difference
    sal_diff := :NEW.SALARY - :OLD.SALARY;

    -- Output employee name and salary details
    DBMS_OUTPUT.PUT_LINE('Employee: ' || :NEW.LAST_NAME);
    DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
    DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in log_salary_changes trigger: ' || SQLERRM);
        -- Optionally log the error to a table
        -- INSERT INTO ERROR_LOG VALUES (SYSDATE, 'log_salary_changes', SQLERRM);
        RAISE; -- Re-raise the exception to prevent the update if desired
END;
/
```



Output

The screenshot shows the Oracle SQL Developer interface. At the top, there are several tabs for different databases: 'banque_system~5', 'plsql2024_class~4', 'System_PDB~1', 'System_PDB~2', 'HR', and 'HR~1'. The main window is divided into two panes. The top pane, titled 'Worksheet', contains a SQL query: `UPDATE EMPLOYEES SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID IN (16, 342);`. The bottom pane, titled 'Script Output', shows the results of the query execution. It starts with 'Trigger execution started', followed by 'Processing row for employee ID: 16'. It then shows the salary update for employee 16 from 10500 to 11500, with a difference of 1000. Next, it processes employee ID 342, updating the salary from 10630 to 11630, also with a difference of 1000. The output concludes with 'Total rows affected: 2' and '2 rows updated.'

```
Worksheet  Query Builder
47 UPDATE EMPLOYEES
48 SET SALARY = SALARY + 1000
49 WHERE EMPLOYEE_ID IN (16, 342);
50

Script Output  x  Query Result  x  Query Result 1  x  Query Result 2  x  Query Result 3  x  Query Result 4  x
Task completed in 0.027 seconds

Trigger execution started
Processing row for employee ID: 16
Updating salary for employee ID: 16 from 10500 to 11500
Updating salary for employee ID: 16 from 10500 to 11500
Employee: Ndahiriwe
Old salary = 10500, New salary = 11500
Difference: 1000
Processing row for employee ID: 342
Updating salary for employee ID: 342 from 10630 to 11630
Updating salary for employee ID: 342 from 10630 to 11630
Total rows affected: 2

2 rows updated.
```

Row-Level Trigger – Optimized

Possible Enhancements (Optional)

1. Log to a Table Instead of Using DBMS_OUTPUT
For persistent tracking:

```
CREATE TABLE SALARY_LOG (  
  EMP_ID NUMBER,  
  LAST_NAME VARCHAR2(100),  
  OLD_SALARY NUMBER,  
  NEW_SALARY NUMBER,  
  SAL_DIFF NUMBER,  
  CHANGED_AT DATE  
);
```



Then modify the trigger: This way, you maintain a full audit trail in a separate table.

```
CREATE OR REPLACE TRIGGER log_salary_changes  
  BEFORE UPDATE OF SALARY ON EMPLOYEES  
  FOR EACH ROW  
  WHEN (NEW.ROLE_ID <> 1)  
  BEGIN  
    INSERT INTO SALARY_LOG (  
      EMP_ID,  
      LAST_NAME,  
      OLD_SALARY,  
      NEW_SALARY,  
      SAL_DIFF,  
      CHANGED_AT  
    )  
    VALUES (  
      :NEW.EMPLOYEE_ID,  
      :NEW.LAST_NAME,  
      :OLD.SALARY,  
      :NEW.SALARY,  
      :NEW.SALARY - :OLD.SALARY,  
      SYSDATE  
    );  
  END;  
/
```

Output

Worksheet | Query Builder

```
46 -- To test the trigger  
47 UPDATE EMPLOYEES  
48 SET SALARY = SALARY + 1000  
49 WHERE EMPLOYEE_ID IN (16, 342);  
50  
51 -- To view the logged changes  
52 SELECT * FROM SALARY_LOG;
```

Script Output x | Query Result x | Query Result 1 x | Query Result 2 x | Query Result 3 x | Query Result 4

SQL | All Rows Fetched: 5 in 0 seconds

	EMP_ID	LAST_NAME	ROLE_ID	OLD_SALARY	NEW_SALARY	SAL_DIFF	CHANGED_AT	CHANGED_BY
1	16	Ndahiriwe	3	5500	6000	500	12-APR-25	ADMINPDB
2	16	Ndahiriwe	3	6000	6500	500	12-APR-25	ADMINPDB
3	16	Ndahiriwe	3	6500	7000	500	12-APR-25	ADMINPDB
4	16	Ndahiriwe	3	7000	7500	500	12-APR-25	ADMINPDB
5	16	Ndahiriwe	3	7500	8500	1000	12-APR-25	ADMINPDB

Table Auditing using DML Triggers in Oracle PL/SQL

Auditing Using Triggers vs Auditing Vault: A Detailed Comparison
Both **triggers** and **auditing vaults** are tools used to track and monitor database activities, but they serve different purposes and operate at distinct levels. Here's a detailed breakdown:

Feature	Auditing Using Triggers	Auditing Vault
Scope	Table/View-specific DML operations	Comprehensive (DML, DDL, and system events)
Performance Impact	Can affect application performance	Minimal impact with optimized configurations
Ease of Implementation	Requires coding and testing for each trigger	Requires configuration but no coding
Customizability	Fully customizable	Limited to predefined templates and policies
Tamper Resistance	Can be bypassed by disabling triggers	Logs are secured and tamper-proof
Cost	No additional cost beyond database licensing	Often part of enterprise solutions with extra fees
Regulatory Compliance	Limited (manual efforts required)	Designed for compliance with built-in tools

I. Scenario Question: Banking Context DDL Trigger

Scenario-Based Question

In a bank's database system, schema-level changes such as creating or altering tables for customer records, transaction logs, or account details must be strictly monitored for compliance and security purposes. You are tasked with implementing a mechanism to track and log all **DDL (Data Definition Language)** operations like CREATE, ALTER, and DROP. This ensures that unauthorized or accidental modifications to critical database structures are detected and auditable.

Objectives:

1. Create an Audit Table

Design a table that logs schema-level events, capturing:

- Event type (CREATE, ALTER, DROP).
- Object name (e.g., customer_accounts, transaction_history).
- User performing the operation (e.g., DB_ADMIN).
- Timestamp of the change.

2. Write a DDL Trigger

Implement a trigger that:

- Fires automatically after any CREATE, ALTER, or DROP operation on the database.
- Inserts event details into the audit table.

3. Test the Solution

Perform test operations relevant to a bank:

- Create a new table for storing loan applications (CREATE TABLE loan_applications...).
- Alter an existing table to add a column for interest rates (ALTER TABLE...).
- Drop an obsolete table for temporary data storage (DROP TABLE temp_data...).

4. Validate the Implementation

Query the audit log to ensure the trigger:

- Captures the operation type and object name (e.g., CREATE TABLE loan_applications).
- Logs the user performing the operation and the time.



Example Use Case:

- A bank's database administrator creates a new table for storing customer feedback.
- The trigger logs this CREATE operation into the audit table, noting the table name, the admin's username, and the timestamp.
- If unauthorized schema changes occur, the audit log provides a detailed trail for investigation and accountability.

Solution DDL Trigger: Banque Scenario

Step 1: Create the Audit Table

The audit table will store details of all CREATE, ALTER, and DROP operations.

```
CREATE TABLE audit_ddl_log (  
  id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  event_type VARCHAR2(50),  
  object_name VARCHAR2(100),  
  operation_user VARCHAR2(50),  
  timestamp DATE  
);
```

Validate: Query the audit log:

```
SELECT * FROM audit_ddl_log;
```

Test Case 2: Altering a Table

Execute the following command:

```
ALTER TABLE customer_feedback ADD feedback_date DATE;
```

Test Case 3: Dropping a Table

Execute the following command:

```
DROP TABLE customer_feedback;
```

Validate: Query the audit log:

```
SELECT * FROM audit_ddl_log  
ORDER BY TIMESTAMP DESC;
```

Step 2: Create the DDL Trigger

This trigger will log relevant details whenever a DDL operation occurs.

```
CREATE OR REPLACE TRIGGER log_ddl_operations  
AFTER CREATE OR ALTER OR DROP ON DATABASE  
BEGIN  
  INSERT INTO audit_ddl_log (event_type, object_name, operation_user, timestamp)  
  VALUES (  
    ORA_SYSEVENT,           -- Captures the type of event (e.g., CREATE, ALTER, DROP)  
    ORA_DICT_OBJ_NAME,      -- Captures the affected object (e.g., table name)  
    SYS_CONTEXT('USERENV', 'SESSION_USER'), -- Captures the user performing the operation  
    SYSDATE                 -- Captures the timestamp of the operation  
  );  
END;  
/
```

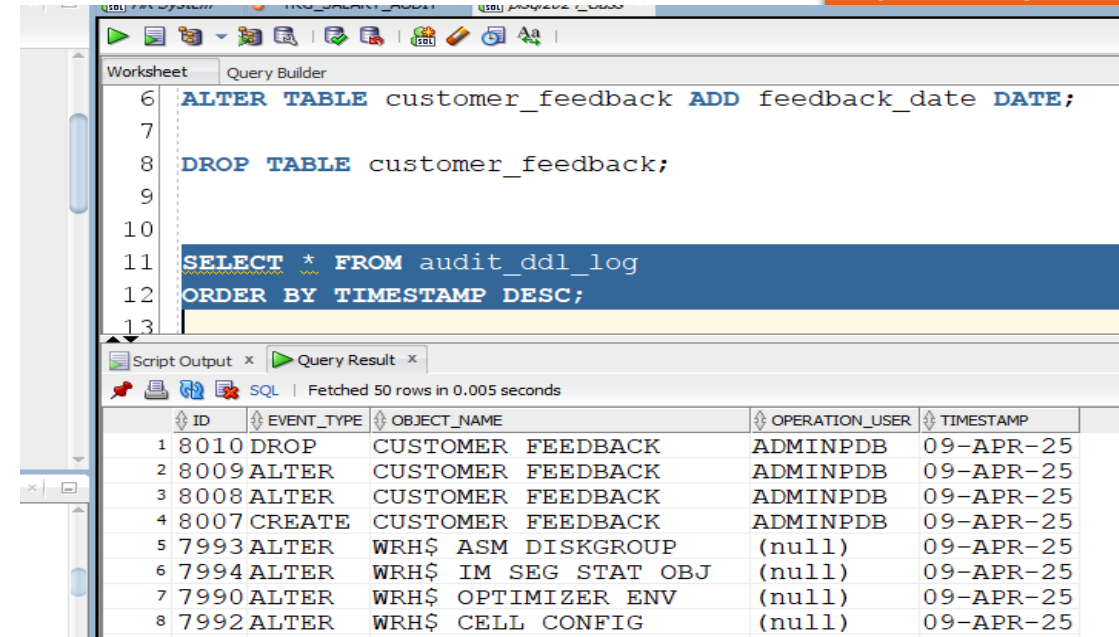
Expected Output

Step 3. Testing

Test Case 1: Creating a Table

Execute the following command:

```
CREATE TABLE customer_feedback (  
  feedback_id NUMBER,  
  feedback_text VARCHAR2(255)  
);
```



	ID	EVENT_TYPE	OBJECT_NAME	OPERATION_USER	TIMESTAMP
1	8010	DROP	CUSTOMER_FEEDBACK	ADMINPDB	09-APR-25
2	8009	ALTER	CUSTOMER_FEEDBACK	ADMINPDB	09-APR-25
3	8008	ALTER	CUSTOMER_FEEDBACK	ADMINPDB	09-APR-25
4	8007	CREATE	CUSTOMER_FEEDBACK	ADMINPDB	09-APR-25
5	7993	ALTER	WRH\$ ASM DISKGROUP	(null)	09-APR-25
6	7994	ALTER	WRH\$ IM SEG STAT OBJ	(null)	09-APR-25
7	7990	ALTER	WRH\$ OPTIMIZER ENV	(null)	09-APR-25
8	7992	ALTER	WRH\$ CELL CONFIG	(null)	09-APR-25

Solution Summary - DDL Trigger: Banque Scenario

Summary of the Solution

1. **Audit Table:** Stores details about schema changes for accountability.
2. **DDL Trigger:** Automatically logs all CREATE, ALTER, and DROP operations.
3. **Testing:** Verified the trigger by performing sample DDL operations and checking the audit log.

This implementation provides a robust mechanism to monitor and log critical schema changes in a bank's database, ensuring security and compliance.

Note:

- **You do not manually call a trigger.** Instead, it gets triggered automatically when the defined DML operation (INSERT, UPDATE, DELETE) happens on the specified table (in this case, EMPLOYEES).
- **To test it,** simply perform the corresponding DML operation (insert, update, or delete) on the EMPLOYEES table.
- **To see the output of the trigger,** ensure DBMS_OUTPUT is enabled and perform the operations.



II. Advanced Trigger Creation Assignment - Salary Change Auditing

Scenario-Based Question :

You are a database developer managing the ICTCHEMBERADMIN.EMPLOYEES table. The HR department wants to **track and audit every change made to employee salaries**. This is to ensure transparency and accountability whenever salary updates occur.

Your Task:

Create a BEFORE UPDATE Trigger on the EMPLOYEES table that:

- Detects changes **only when the salary is modified**.
- Inserts a record into a new audit table called SALARY_AUDIT **every time** an employee's salary is updated.

The inserted record in SALARY_AUDIT should capture the following information:

- EMPLOYEE_ID – The ID of the employee whose salary was changed.
- OLD_SALARY – The salary value **before the update**.
- NEW_SALARY – The salary value **after the update**.
- CHANGE_DATE – The date and time when the change occurred (SYSDATE).
- USERNAME – The database user who made the change (USER).
- SESSION_ID – The session ID of the user who made the change (SYS_CONTEXT('USERENV', 'SESSIONID')).

Additionally

- **Create a stored procedure** to perform a salary update that:
 - Accepts employee_id and new_salary as input parameters.
 - Performs the update to the EMPLOYEES table.
 - Includes **exception handling** to raise an error when the employee_id does not exist or when any unexpected error occurs.

Solution approach – Salary Audit

```
/*  
Step 1: Create the SALARY_AUDIT Table  
*/  
--Step 1: Create the SALARY_AUDIT Table  
CREATE TABLE salary_audit (  
    employee_id  NUMBER(6),           -- Employee's ID  
    old_salary   NUMBER(8,2),         -- Old salary value  
    new_salary   NUMBER(8,2),         -- New salary value  
    change_date  DATE,                -- Date (stores date and time)  
    timestamp_change TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Timestamp for change with default value  
    username     VARCHAR2(30),        -- Username of the person making the change  
    session_id   VARCHAR2(30),        -- Session ID of the user making the change  
);
```

Step 2: Create the Trigger

- Name the trigger: `TRG_SALARY_AUDIT`
- It should fire **only when the salary is actually changed**
- Type of trigger:
`BEFORE UPDATE ON EMPLOYEES FOR EACH ROW`
- Inside the trigger body, compare `:OLD.salary` and `:NEW.salary`
- If they are different, insert a new record into `SALARY_AUDIT` with the required fields

Deliverables:

- SQL script for creating the `SALARY_AUDIT` table
- SQL script for the `TRG_SALARY_AUDIT` trigger
- A brief explanation (2–4 lines) of how your trigger works

Solution – Trigger Salary Audit

Step 3: Create the Trigger

```
CREATE OR REPLACE TRIGGER trg_salary_audit
BEFORE UPDATE ON employees
FOR EACH ROW
WHEN (OLD.salary != NEW.salary) -- Only fire if salary is being changed
BEGIN
    BEGIN
        -- Insert the salary change into the audit table
        INSERT INTO salary_audit (
            employee_id,
            old_salary,
            new_salary,
            change_date,
            username,
            session_id
        )
        VALUES (
            :OLD.employee_id,
            :OLD.salary,
            :NEW.salary,
            SYSDATE, -- Current system date and time
            USER,    -- Current database user
            SYS_CONTEXT('USERENV', 'SESSIONID') -- Current session ID
        );
    EXCEPTION
        WHEN OTHERS THEN
            -- Handle unexpected errors silently to avoid interrupting the original UPDATE
            -- Optionally, log error into an error_log table (not shown here)
            NULL; -- Replace with logging if desired
    END;
END;
/
```

Solution – Procedure to update salary

Step 3: Procedure to update salary

```
CREATE OR REPLACE PROCEDURE update_employee_salary (  
    p_employee_id IN NUMBER,  
    p_new_salary IN NUMBER  
) AS  
BEGIN  
    -- Update employee salary in the employees table  
    UPDATE employees  
    SET salary = p_new_salary  
    WHERE employee_id = p_employee_id;  
  
    -- Ensure that the update statement actually modifies a row  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE_APPLICATION_ERROR(-20003, 'No employee found with ID ' || p_employee_id);  
    END IF;  
  
END;  
/
```

```
/*  
Call the procedure and input id and salary to be changed.  
*/  
-- Call the procedure  
BEGIN  
    UPDATE_EMPLOYEE_SALARY(101, 70500);  
END;  
/
```

```
56 /*  
57 Call the procedure and input id and salary to be changed.  
58 */  
59 -- Call the procedure  
60 BEGIN  
61     UPDATE_EMPLOYEE_SALARY(102, 70500);  
62 END;  
63 /  
64  
65 SELECT * FROM salary_audit;
```

Test Procedure & Trigger → Output

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x									
SQL All Rows Fetched: 2 in 0.004 seconds									
	EMPLOYEE_ID	OLD_SALARY	NEW_SALARY	CHANGE_DATE	TIMESTAMP_CHANGE	USERNAME	SESSION_ID		
1	101	4500	7500	09-APR-25	09-APR-25 08.08.26.274000000	PM BASICSADMIN	361085		
2	102	17000	70500	09-APR-25	09-APR-25 08.29.19.363000000	PM BASICSADMIN	361085		

III. Creating a DML Trigger for Auditing Allowances Table Operations

Scenario-Based Question:

You are tasked with implementing an **audit mechanism** to log all **DML (Data Manipulation Language) operations** (INSERT, UPDATE, DELETE) on critical tables in a banking system database. This ensures accountability and transparency by tracking changes made to key tables.

Tables to be Audited:

- ALLOWANCES
- ATTENDANCE
- COUNTRIES
- DEPARTMENTS
- EMPLOYEES
- ROLES

Objectives:

1. Create an Audit Table:

- The table should capture the following for each DML operation:
 - **operation_type**: INSERT, UPDATE, DELETE
 - **object_name**: Table name
 - **operation_user**: The user who performed the operation
 - **timestamp**: When the operation occurred
 - **old_data**: Data before the operation (only for UPDATE and DELETE)
 - **new_data**: Data after the operation (only for INSERT and UPDATE)

2. Create a Trigger for Logging DML Operations:

- Write a trigger for the ALLOWANCES table (and similar ones for others), which fires **after** INSERT, UPDATE, or DELETE.
- The trigger should insert logs into the audit table, capturing **old data** for DELETE/UPDATE and **new data** for INSERT/UPDATE operations.

3. Stored Procedures for DML Operations:

- Write **UPDATE** and **DELETE** procedures for the ALLOWANCES table with exception handling.
- If an error occurs, log it in an **error log table**.

4. Test the Implementation:

- Perform INSERT, UPDATE, and DELETE operations on ALLOWANCES.
- Verify that the audit table logs the operations correctly.
- Check the error log table for any logged errors.

Deliverables:

- SQL scripts to:
 - Create the **audit table**.
 - Implement **triggers** for DML operations.
 - Write **stored procedures** for UPDATE and DELETE operations.
 - Test results showing the audit logs and error logs.

Evaluation Criteria:

- **Correctness**: Does the trigger capture all DML operations with appropriate details?
- **Error Handling**: Are errors correctly logged in the error log table?
- **Efficiency**: Are the procedures optimized for performance while handling exceptions?



Solution - DML Trigger - Auditing Allowances Operations (1/4)

`SYS_CONTEXT('USERENV', 'SESSION_USER')` is an Oracle SQL function used to retrieve information about the current session, specifically the session user (i.e., the user currently connected to the database).

```
/*
Step 1: Create the Audit Table
First, let's create an audit table that logs the changes made to any of these tables.
*/
```

```
CREATE TABLE audit_dml_log (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY, -- Unique identifier
  operation_type VARCHAR2(50), -- Type of operation (INSERT, UPDATE, DELETE)
  object_name VARCHAR2(100), -- Name of the affected table
  operation_user VARCHAR2(50), -- User who performed the operation
  timestamp DATE, -- Date and time of the operation
  old_data VARCHAR2(4000), -- Old data (for UPDATE and DELETE)
  new_data VARCHAR2(4000) -- New data (for INSERT and UPDATE)
);
```

```
/*
Step 2: Create the DML Trigger
Now, let's create a DML Trigger for each table (e.g., ALLOWANCES, ATTENDANCE, COUNTRIES, DEPARTMENTS,
EMPLOYEES, ROLES). This trigger will capture the changes made (insertion, update, deletion) to any of the tables and log
those changes in the audit_dml_log table.
*/
```

```
CREATE OR REPLACE TRIGGER log_dml_operations
AFTER INSERT OR UPDATE OR DELETE ON ALLOWANCES -- You can change this table name dynamically if needed
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, new_data)
    VALUES ('INSERT', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:NEW.ALLOWANCE_AMOUNT);
  ELSIF UPDATING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data, new_data)
    VALUES ('UPDATE', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:OLD.ALLOWANCE_AMOUNT, :NEW.ALLOWANCE_AMOUNT);
  ELSIF DELETING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data)
    VALUES ('DELETE', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:OLD.ALLOWANCE_AMOUNT);
  END IF;
END;
```

Solution - DML Trigger - Auditing Attendance Operations (2/3)

```
/*
```

Step 3: Repeat for Other Tables

You can repeat the process for other tables (ATTENDANCE, COUNTRIES, DEPARTMENTS, EMPLOYEES, ROLES) by changing the AFTER INSERT OR UPDATE OR DELETE ON part of the trigger, and modifying the field names in the :NEW and :OLD references according to the specific columns in those tables.

For example, for the ATTENDANCE table, the trigger would look like this:

```
*/
```

```
CREATE OR REPLACE TRIGGER log_dml_operations_attendance
AFTER INSERT OR UPDATE OR DELETE ON ATTENDANCE
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, new_data)
        VALUES ('INSERT', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :NEW.STATUS);
    ELSIF UPDATING THEN
        INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data, new_data)
        VALUES ('UPDATE', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :OLD.STATUS,
:NEW.STATUS);
    ELSIF DELETING THEN
        INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data)
        VALUES ('DELETE', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :OLD.STATUS);
    END IF;
END;
/
```

Test – Update Allowances Operations (2/3)

```
CREATE OR REPLACE PROCEDURE update_allowance(  
    p_allowance_id IN NUMBER,  
    p_new_amount   IN NUMBER  
) AS  
    e_id_not_found EXCEPTION;  
    PRAGMA EXCEPTION_INIT(e_id_not_found, -20001);  
BEGIN  
    UPDATE ALLOWANCES  
    SET ALLOWANCE_AMOUNT = p_new_amount  
    WHERE ALLOWANCE_ID = p_allowance_id;  
  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Allowance ID does not exist.');    END IF;  
  
    COMMIT;  
EXCEPTION  
    WHEN e_id_not_found THEN  
        RAISE;  
    WHEN OTHERS THEN  
        RAISE;  
END;  
/
```

update_allowance with exception handling

--Call update allowance

```
BEGIN  
    update_allowance(12, 6000);  
END;  
/
```

Test – Delete Allowances & Update Attendance (3/4)

```
CREATE OR REPLACE PROCEDURE delete_allowance(  
    p_allowance_id IN ALLOWANCE_ID NUMBER  
  
    ) AS  
BEGIN  
    DELETE FROM ALLOWANCES  
    WHERE ALLOWANCE_ID = p_allowance_id;  
  
    COMMIT;  
EXCEPTION  
    WHEN OTHERS THEN  
        INSERT INTO audit_dml_error_log (error_message, error_time, object_name, operation_user)  
        VALUES (SQLERRM, SYSDATE, 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'));  
        RAISE;  
END;  
/
```

delete_allowance

```
BEGIN  
    delete_allowance(12);  
END;  
/
```

Update attendance

```
CREATE OR REPLACE PROCEDURE update_attendance(  
    p_attendance_id IN NUMBER,  
    p_new_status IN VARCHAR2  
    ) AS  
BEGIN  
    UPDATE ATTENDANCE  
    SET STATUS = p_new_status  
    WHERE ATTENDANCE_ID = p_attendance_id;  
  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Attendance ID does not exist.');
```

```
--Call update_allowance  
BEGIN  
    update_attendance(101, 'Present');  
END;  
/
```

Test – Delete Attendance (4/4)

```
CREATE OR REPLACE PROCEDURE delete_attendance(  
    p_attendance_id IN NUMBER  
) AS  
BEGIN  
    DELETE FROM ATTENDANCE  
    WHERE ATTENDANCE_ID = p_attendance_id;  
  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Error: Attendance ID does not exist.');    END IF;  
  
    COMMIT;  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE_APPLICATION_ERROR(-20002, 'Unexpected error occurred while deleting attendance: ' ||  
SQLERRM);  
END;  
/
```

delete_attendance

```
BEGIN  
    delete_attendance(101);  
END;  
/
```

Test – Insert Data in Allowances & Results: Audit Tables (3/3)

```
/*
```

Step 4: Test the Trigger

Let's perform some operations on the ALLOWANCES table to verify that the trigger is working as expected.

Test Case 1: Inserting a Row

```
*/
```

```
INSERT INTO ALLOWANCES (ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
```

```
VALUES ( 12, 2, 5000, TO_DATE('2024-11-01', 'YYYY-MM-DD'), 'Y', 'Performance-based allowance');
```

```
INSERT INTO ALLOWANCES (ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
```

```
VALUES (24, 3, 6000, TO_DATE('2024-11-01', 'YYYY-MM-DD'), 'Y', 'Team lead allowance');
```

```
INSERT INTO ALLOWANCES (ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
```

```
VALUES (25, 4, 4500, TO_DATE('2024-11-01', 'YYYY-MM-DD'), 'Y', 'Department bonus');
```

```
INSERT INTO ALLOWANCES (ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
```

```
VALUES (26, 5, 5200, TO_DATE('2024-11-01', 'YYYY-MM-DD'), 'Y', 'Special project reward');
```

Output

```
27 Check the audit log:
28 */
29 SELECT * FROM audit_dml_log;
```

ID	OPERATION_TYPE	OBJECT_NAME	OPERATION_USER	TIMESTAMP	OLD_DATA	NEW_DATA
1	41 INSERT	ALLOWANCES	ADMINPDB	09-APR-25	(null)	5000
2	42 INSERT	ALLOWANCES	ADMINPDB	09-APR-25	(null)	6000
3	43 INSERT	ALLOWANCES	ADMINPDB	09-APR-25	(null)	4500
4	44 INSERT	ALLOWANCES	ADMINPDB	09-APR-25	(null)	5200
5	45 UPDATE	ALLOWANCES	ADMINPDB	10-APR-25	5000	252545
6	46 UPDATE	ALLOWANCES	ADMINPDB	10-APR-25	500	2555000
7	61 UPDATE	ATTENDANCE	ADMINPDB	10-APR-25	Present	Present
8	62 DELETE	ATTENDANCE	ADMINPDB	10-APR-25	Present	(null)

```
/*
```

Check the audit log:

```
*/
```

```
SELECT * FROM audit_dml_log;
```

-- DML logs

```
SELECT * FROM audit_dml_log ORDER BY timestamp DESC;
```

IV. Database Error Logging with Trigger – Practical Task

Scenario-Based Question

You are working as a Database Administrator in an organization where it is critical to log all DML (Data Manipulation Language) errors for audit and debugging purposes. Your goal is to ensure that whenever a DML error occurs (e.g., due to a constraint violation), a trigger automatically logs the error details into a table called `dml_error_log`.

Task Overview

- **Review the error log table creation script:**
Understand the structure of the `dml_error_log` table. This table is used to store information about DML errors, such as the error message, time, object name, action type, session ID, user, and SQL text (if available).
- **Understand the server error trigger:**
Examine the `trg_log_dml_errors` trigger that fires automatically after a server error occurs. It captures error details and inserts them into the `dml_error_log` table.
- **Test the setup by generating a DML error:**
Now simulate a real-world scenario by executing an invalid SQL statement (for example, inserting a duplicate value into a table with a PRIMARY KEY or UNIQUE constraint). This error should be captured and logged automatically by the trigger.

Instructions

- Run the table and trigger creation script to set up the environment.
- Choose or create a sample table for testing.



Solution : Error record Table & Index created

```
CREATE TABLE dml_error_log (  
  log_id      NUMBER GENERATED ALWAYS AS IDENTITY,  
  error_message VARCHAR2(4000),  
  error_time   TIMESTAMP DEFAULT SYSTIMESTAMP,  
  object_name  VARCHAR2(128),  
  action       VARCHAR2(30),  
  session_id   NUMBER,  
  username     VARCHAR2(128),  
  sql_text     CLOB,  
  CONSTRAINT pk_dml_error_log PRIMARY KEY (log_id)  
);  
  
-- Create index on commonly searched columns  
CREATE INDEX idx_dml_error_log_time ON dml_error_log(error_time);  
CREATE INDEX idx_dml_error_log_object ON dml_error_log(object_name);  
  
-- Create a comment on the table  
COMMENT ON TABLE dml_error_log IS 'Logs DML errors from database operations';  
  
-- Create comments on columns  
COMMENT ON COLUMN dml_error_log.log_id IS 'Primary key - automatically generated';  
COMMENT ON COLUMN dml_error_log.error_message IS 'Oracle error message text';  
COMMENT ON COLUMN dml_error_log.error_time IS 'Timestamp when the error occurred';  
COMMENT ON COLUMN dml_error_log.object_name IS 'Name of the table or object that caused the error';  
COMMENT ON COLUMN dml_error_log.action IS 'Type of DML operation (INSERT, UPDATE, DELETE, ALTER)';  
COMMENT ON COLUMN dml_error_log.session_id IS 'Database session ID where the error occurred';  
COMMENT ON COLUMN dml_error_log.username IS 'Database username that executed the statement';  
COMMENT ON COLUMN dml_error_log.sql_text IS 'SQL statement that caused the error (when available)';  
  
-- Grant permissions as needed  
-- GRANT SELECT ON dml_error_log TO reporting_role;  
-- GRANT INSERT ON dml_error_log TO app_role;
```

Trigger Definition : Error Records

```
CREATE OR REPLACE TRIGGER trg_log_dml_errors
--fire automatically after any error occurs / AFTER SERVERERROR Trigger activates after an error occurs
AFTER SERVERERROR ON SCHEMA
DECLARE
    v_error_stack VARCHAR2(4000);
    v_object_name VARCHAR2(128);
    v_action VARCHAR2(30);
BEGIN
    -- Get the error information
    v_error_stack := DBMS_UTILITY.FORMAT_ERROR_STACK;

    -- Determine which object and action caused the error / For demonstration, we'll check the current SQL statement

    IF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'ATTENDANCE') > 0 THEN
        v_object_name := 'ATTENDANCE';
    ELSIF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'ALLOWANCES') > 0 THEN
        v_object_name := 'ALLOWANCES';
    ELSE
        v_object_name := 'UNKNOWN';
    END IF;

    -- Determine the action type
    IF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'INSERT') > 0 THEN
        v_action := 'INSERT';
    ELSIF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'UPDATE') > 0 THEN
        v_action := 'UPDATE';
    ELSIF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'DELETE') > 0 THEN
        v_action := 'DELETE';
    ELSIF INSTR(UPPER(DBMS_UTILITY.FORMAT_CALL_STACK), 'ALTER') > 0 THEN
        v_action := 'ALTER';
    ELSE
        v_action := 'UNKNOWN';
    END IF;

    -- Insert into error log table
    INSERT INTO dml_error_log (
        error_message,
        error_time,
        object_name,
        action,
        session_id,
        username
    ) VALUES (
        v_error_stack,
        SYSTIMESTAMP,
        v_object_name,
        v_action,
        SYS_CONTEXT('USERENV', 'SID'),
        SYS_CONTEXT('USERENV', 'SESSION_USER')
    );
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- If error logging itself fails, write to server log
        DBMS_OUTPUT.PUT_LINE('Error logging failed: ' || SQLERRM);
END;
/
```

Basic Setup Testing

Output

Welcome Page x HR x HR~1 x System_PDB x AUDIT_DDL_LOG x ALLOWANCES_AUDIT_TRIGGER x HR~2 x									
Worksheet Query Builder									
Select * from dml_error_log;									
Query Result x									
All Rows Fetched: 131 in 0.027 seconds									
LOG_ID	ERROR_MESSAGE	ERROR_TIME	OBJECT_NAME	ACTION	SESSION_ID	USERNAME	SQL_TEXT		
115	107 ORA-04095: trigger 'TRG_INCREASE_SALARY' already exists on another table, cannot...	12-APR-25 08.04.07.120000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
116	108 ORA-04095: trigger 'TRG_INCREASE_SALARY' already exists on another table, cannot...	12-APR-25 08.04.48.243000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
117	109 ORA-01400: cannot insert NULL into ("ADMINPDB"."EMPLOYEES_TEST"."FIRST_NAME")	12-APR-25 08.08.53.862000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
118	110 ORA-01400: cannot insert NULL into ("ADMINPDB"."EMPLOYEES_TEST"."FIRST_NAME")	12-APR-25 08.09.10.860000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
119	111 ORA-04043: object GREETING_PKG does not exist	12-APR-25 10.33.53.558000000 PM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
120	112 ORA-04063: package body "ADMINPDB.EMPLOYEE_PKG" has errorsORA-06508: PL/SQL: cou...	13-APR-25 12.05.06.320000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
121	113 ORA-04063: package body "ADMINPDB.EMPLOYEE_PKG" has errorsORA-06508: PL/SQL: cou...	13-APR-25 12.05.10.067000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
122	114 ORA-24256: EXPAND_SQL_TEXT failed with ORA-00904: "P_EMP_ID": invalid identifier...	13-APR-25 12.43.06.761000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
123	115 ORA-00955: name is already used by an existing object	13-APR-25 09.11.53.185000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
124	116 ORA-00955: name is already used by an existing object	13-APR-25 09.11.58.854000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
125	117 ORA-24256: EXPAND_SQL_TEXT failed with ORA-00904: "ID": invalid identifierORA-06...	13-APR-25 10.21.00.819000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
126	118 ORA-01917: user or role 'SOME_USER' does not exist	13-APR-25 10.35.23.005000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
127	119 ORA-00955: name is already used by an existing object	13-APR-25 10.44.49.942000000 AM	UNKNOWN	UNKNOWN	494	ADMINPDB	(null)		
128	166 ORA-00955: name is already used by an existing object	16-APR-25 06.04.55.491000000 PM	UNKNOWN	UNKNOWN	373	ADMINPDB	(null)		
129	167 ORA-00942: table or view does not exist	16-APR-25 06.04.55.594000000 PM	UNKNOWN	UNKNOWN	373	ADMINPDB	(null)		

System Event Trigger Definition

A **system event trigger** in a database is used to automatically execute a PL/SQL block of code in response to certain predefined system-level events. These events are not associated with any specific table but are linked to database operations like user logins, database startup or shutdown, and DDL (Data Definition Language) changes.

Common Events for System Triggers:

- **LOGON:** Triggered when a user logs into the database.
- **LOGOFF:** Triggered when a user logs off from the database.
- **STARTUP:** Triggered when the database is started.
- **SHUTDOWN:** Triggered when the database is shut down.
- **AFTER SERVERERROR:** Triggered after a server error occurs.
- **AFTER DDL:** Triggered after a DDL statement (e.g., CREATE, ALTER, DROP) is executed.
- **BEFORE DDL:** Triggered before a DDL statement is executed.

```
CREATE OR REPLACE TRIGGER trigger_name
-- Specify the timing of the trigger (BEFORE or AFTER the event)
AFTER | BEFORE

-- Specify the system event to monitor
[STARTUP | SHUTDOWN | LOGON | LOGOFF | DDL | SERVERERROR]

-- Specify the scope of the trigger (DATABASE for all users, SCHEMA for the current
schema)
ON [DATABASE | SCHEMA]

BEGIN
-- Add the PL/SQL code here to define the trigger's action
-- e.g., log information, modify session, etc.
END;
/
```

Explanation of the Components:

- **CREATE OR REPLACE TRIGGER trigger_name:** Defines a new trigger or replaces an existing trigger with the same name.
- **AFTER | BEFORE:** Specifies whether the trigger should fire **before** or **after** the event occurs.
- **[STARTUP | SHUTDOWN | LOGON | LOGOFF | DDL | SERVERERROR]:** Specifies the database event that will activate the trigger.
- **ON [DATABASE | SCHEMA]:** Defines the scope of the trigger:
 - **DATABASE:** Trigger applies to all users of the database.
 - **SCHEMA:** Trigger applies only to the schema of the user creating the trigger.

Classification of SQL Operations

Activity	SQL Category	Description
CREATE TABLE, ALTER TABLE, etc.	DDL	Defines and modifies the structure of database objects
INSERT, UPDATE, DELETE	DML (Data Manipulation Language)	Changes data inside tables
GRANT, REVOKE	DCL (Data Control Language)	Manages permissions and access controls
Login/Logoff auditing	DBMS auditing or security features	Typically handled by database configuration or logs

I. Logging User Logins

This trigger captures login events and logs them into a custom USER_LOG table.

Create Audit Table for Login Logs

```
-- Create a table to log user activity
CREATE TABLE USER_LOG (
  LOG_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, -- Unique identifier for each log entry
  USERNAME VARCHAR2(50), -- Stores the username of the logged-in user
  SESSION_ID NUMBER, -- Tracks the session ID of the user
  LOGON_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Records the logon time (defaults to the current time)
);
```

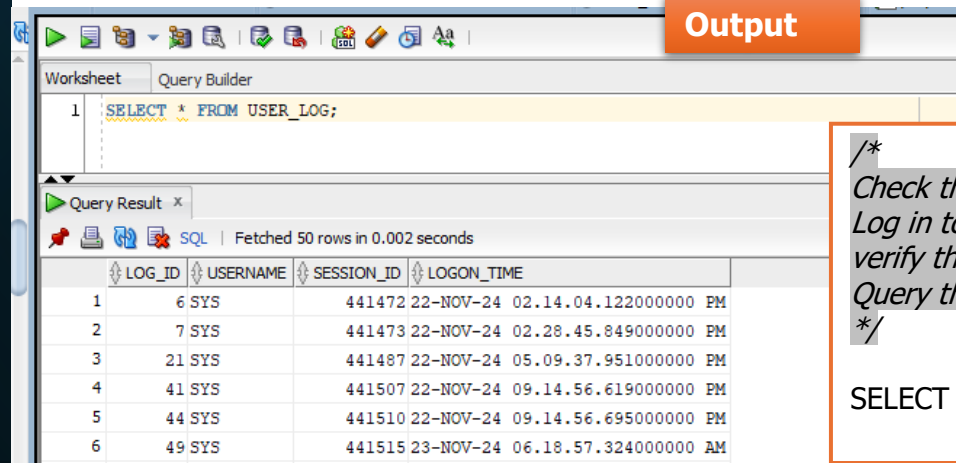
Create a trigger

```
-- Create or replace a trigger named 'trg_logon_audit'
CREATE OR REPLACE TRIGGER trg_logon_audit

-- Specify that the trigger fires AFTER a LOGON event on the entire DATABASE
AFTER LOGON ON DATABASE
BEGIN
  -- Insert a record into the USER_LOG table for every user logon
  INSERT INTO USER_LOG (USERNAME, SESSION_ID, LOGON_TIME)

  -- USER: Captures the username of the logged-in user
  -- SYS_CONTEXT('USERENV', 'SESSIONID'): Retrieves the current session ID
  -- SYSTIMESTAMP: Captures the current timestamp for the logon
  VALUES (USER, SYS_CONTEXT('USERENV', 'SESSIONID'), SYSTIMESTAMP);
END trg_logon_audit;
/
```

Output



The screenshot shows a database interface with a query result table. The query is 'SELECT * FROM USER_LOG;'. The result table has columns LOG_ID, USERNAME, SESSION_ID, and LOGON_TIME. It displays 6 rows of login data.

	LOG_ID	USERNAME	SESSION_ID	LOGON_TIME
1	6	SYS	441472	22-NOV-24 02.14.04.122000000 PM
2	7	SYS	441473	22-NOV-24 02.28.45.849000000 PM
3	21	SYS	441487	22-NOV-24 05.09.37.951000000 PM
4	41	SYS	441507	22-NOV-24 09.14.56.619000000 PM
5	44	SYS	441510	22-NOV-24 09.14.56.695000000 PM
6	49	SYS	441515	23-NOV-24 06.18.57.324000000 AM

```
/*
Check the USER_LOG Table:
Log in to the database as the user who created the trigger and
verify the records.
Query the USER_LOG table:
*/
```

```
SELECT * FROM USER_LOG;
```

II. Logging User Logins

This trigger tracks all Data Definition Language (DDL) operations (e.g., CREATE, ALTER, DROP) performed in the schema.

```
CREATE TABLE DDL_LOG (  
    LOG_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    USERNAME VARCHAR2(50),  
    SCHEMA_NAME VARCHAR2(50),  
    EVENT_TYPE VARCHAR2(50),  
    OBJECT_NAME VARCHAR2(100),  
    TIMESTAMP TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Create Audit Table for DDL Logs

```
-- Create or replace a trigger named 'trg_ddl_audit'  
CREATE OR REPLACE TRIGGER trg_ddl_audit
```

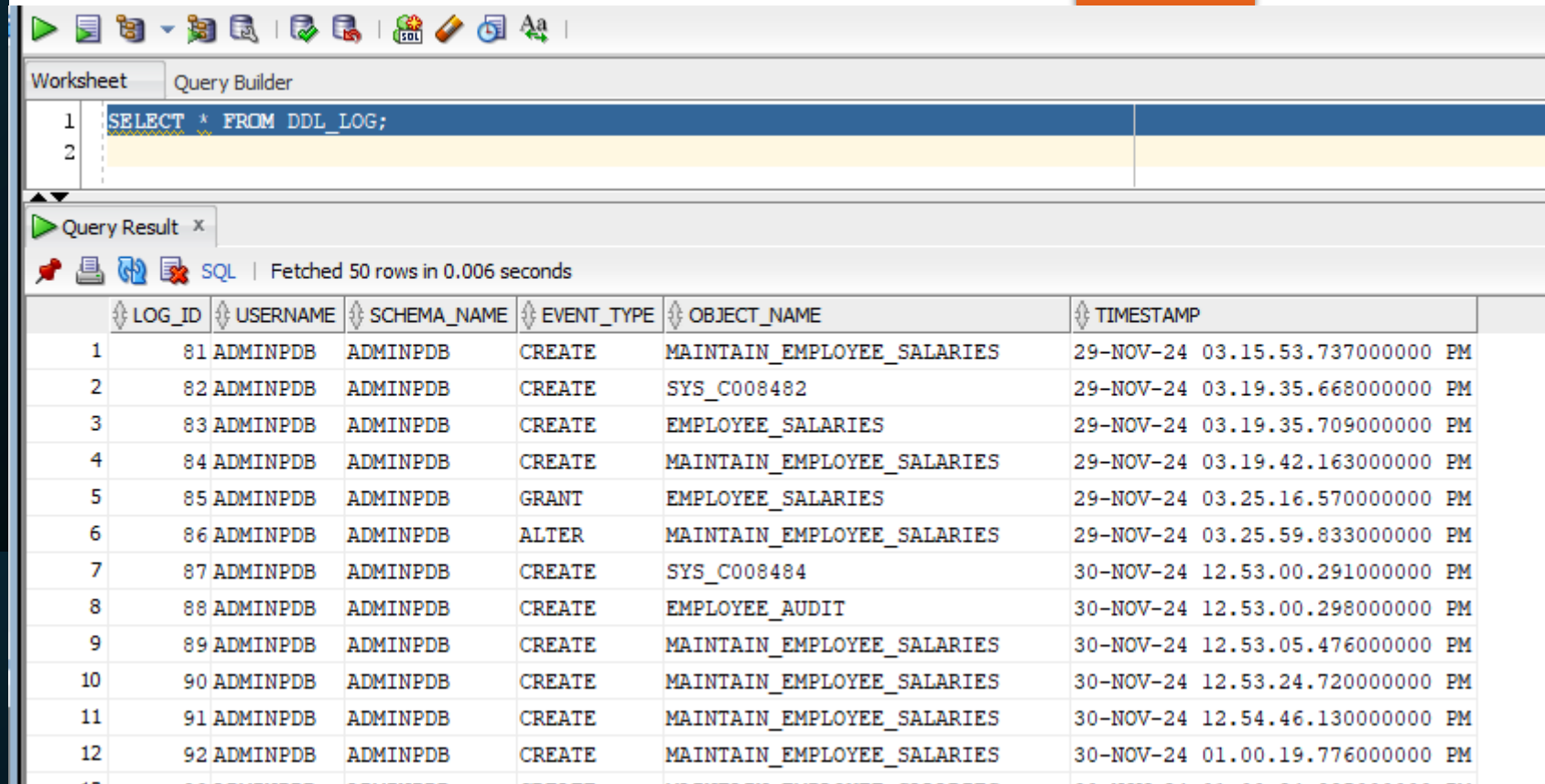
Create a trigger

```
-- Specify that the trigger fires AFTER any DDL (Data Definition Language) operation  
on the current schema  
AFTER DDL ON SCHEMA  
BEGIN  
    -- Insert a record into the DDL_LOG table for every DDL operation  
    INSERT INTO DDL_LOG (  
        USERNAME, -- The user performing the DDL operation  
        SCHEMA_NAME, -- The schema where the DDL operation is being executed  
        EVENT_TYPE, -- The type of DDL event (e.g., CREATE, ALTER, DROP)  
        OBJECT_NAME, -- The name of the object affected by the DDL event (e.g.,  
table, view, etc.)  
        TIMESTAMP -- The timestamp when the DDL operation occurred  
    )  
    VALUES (  
        USER, -- Built-in function returning the username  
        SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA'), -- Retrieves the current  
schema name  
        ORA_SYSEVENT, -- Built-in variable for the type of DDL event  
(e.g., CREATE, ALTER, DROP)  
        ORA_DICT_OBJ_NAME, -- Built-in variable for the name of the  
object involved in the DDL operation  
        SYSTIMESTAMP -- Captures the current date and time  
    );  
END trg_ddl_audit;  
/
```

Solution Logging DDL Change – Output

```
/*  
Step 1: Ensure the DDL_LOG Table Exists  
Before testing, make sure the DDL_LOG table is created and ready to store audit records:  
*/  
SELECT * FROM DDL_LOG;
```

Output



	LOG_ID	USERNAME	SCHEMA_NAME	EVENT_TYPE	OBJECT_NAME	TIMESTAMP
1	81	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	29-NOV-24 03.15.53.737000000 PM
2	82	ADMINPDB	ADMINPDB	CREATE	SYS_C008482	29-NOV-24 03.19.35.668000000 PM
3	83	ADMINPDB	ADMINPDB	CREATE	EMPLOYEE_SALARIES	29-NOV-24 03.19.35.709000000 PM
4	84	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	29-NOV-24 03.19.42.163000000 PM
5	85	ADMINPDB	ADMINPDB	GRANT	EMPLOYEE_SALARIES	29-NOV-24 03.25.16.570000000 PM
6	86	ADMINPDB	ADMINPDB	ALTER	MAINTAIN_EMPLOYEE_SALARIES	29-NOV-24 03.25.59.833000000 PM
7	87	ADMINPDB	ADMINPDB	CREATE	SYS_C008484	30-NOV-24 12.53.00.291000000 PM
8	88	ADMINPDB	ADMINPDB	CREATE	EMPLOYEE_AUDIT	30-NOV-24 12.53.00.298000000 PM
9	89	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	30-NOV-24 12.53.05.476000000 PM
10	90	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	30-NOV-24 12.53.24.720000000 PM
11	91	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	30-NOV-24 12.54.46.130000000 PM
12	92	ADMINPDB	ADMINPDB	CREATE	MAINTAIN_EMPLOYEE_SALARIES	30-NOV-24 01.00.19.776000000 PM

III. Logging Shutdown/restarts Events

```
-- Create the logging table
CREATE TABLE shutdown_log (
  log_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
  event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  event_type VARCHAR2(100),
  event_message VARCHAR2(4000)
);
```

```
-- Create a before shutdown trigger (fixed version)
CREATE OR REPLACE TRIGGER trg_before_shutdown
BEFORE SHUTDOWN ON DATABASE
BEGIN
  INSERT INTO shutdown_log (event_type, event_message)
  VALUES ('PLANNED_SHUTDOWN', 'Database shutdown initiated through normal command');
  COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    -- Try to log the error
    BEGIN
      INSERT INTO shutdown_log (event_type, event_message)
      VALUES ('SHUTDOWN_ERROR', 'Error during shutdown trigger: ' ||
        DBMS_UTILITY.FORMAT_ERROR_STACK);
      COMMIT;
    EXCEPTION
      WHEN OTHERS THEN
        NULL; -- Silent failure as last resort
    END;
END;
/
```

A pre-shutdown trigger

A pre-shutdown trigger

- System state views monitoring
- Alert log monitoring

```
-- Create an instance startup trigger to log restarts
CREATE OR REPLACE TRIGGER trg_after_startup
AFTER STARTUP ON DATABASE
BEGIN
  INSERT INTO shutdown_log (event_type, event_message)
  VALUES ('STARTUP', 'Database started up');
  COMMIT;
END;
/
```

Trigger log monitoring

Manual Test with DBA Privileges

```
-- Connect as user with shutdown privileges (like SYSDBA)
sqlus username/password AS SYSDBA
```

```
-- Initiate a shutdown
SHUTDOWN IMMEDIATE;
```

Output

```
-- After restarting the database
STARTUP;
```

```
-- Check the log table
SELECT * FROM shutdown_log ORDER BY event_time DESC;
```

```
C:\Users\Eric>sqlplus / as sysdba

SQL*Plus: Release 21.0.0.0.0 - Production on Fri Apr 11 06:32:10 2025
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to an idle instance.

SQL> startup;
ORACLE instance started.

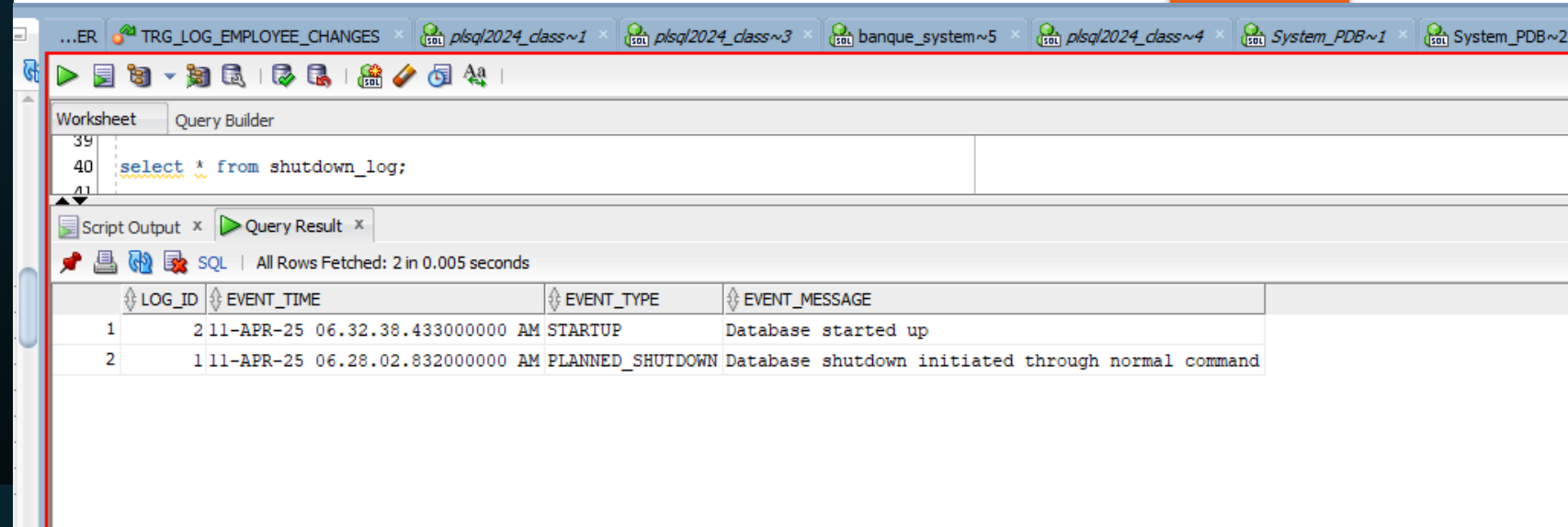
Total System Global Area 2147482528 bytes
Fixed Size 9857952 bytes
Variable Size 1392508928 bytes
Database Buffers 738197504 bytes
Redo Buffers 6918144 bytes
Database mounted.
Database opened.
SQL> show con_name;

CON_NAME
-----
CDB$ROOT
SQL> |
```

Manual Test with DBA Privileges - Output

```
-- Check the log table  
SELECT * FROM shutdown_log ORDER BY event_time DESC;
```

Output



Worksheet | Query Builder

39
40 `select * from shutdown_log;`
41

Script Output x | Query Result x

SQL | All Rows Fetched: 2 in 0.005 seconds

LOG_ID	EVENT_TIME	EVENT_TYPE	EVENT_MESSAGE
1	2 11-APR-25 06.32.38.433000000 AM	STARTUP	Database started up
2	1 11-APR-25 06.28.02.832000000 AM	PLANNED_SHUTDOWN	Database shutdown initiated through normal command

IV. Trigger Event: Enforcing HR Transaction Restrictions During Non-Working Hours

Scenario-Based Question:

The **Human Resources Department** of a large organization manages allowances for its employees. As part of the organizational policy, it is required that any changes to the **ALLOWANCES** table—whether it be inserting new allowances, updating existing records, or deleting them—should only be allowed during official working hours (from **6:00 AM to 6:00 PM**). This policy is designed to ensure that only authorized personnel, during working hours, can make changes to employee allowances. If an attempt is made to insert, update, or delete records outside of these hours (e.g., during the night or weekends), the system should automatically block such attempts and raise an appropriate error message to inform the user of the restriction.

Task:

As a database developer, you are tasked with implementing a PL/SQL trigger that enforces this policy on the **ALLOWANCES** table. The trigger should:

1. Fire **before any INSERT, UPDATE, or DELETE** operation on the **ALLOWANCES** table.
2. Check the system's current time to ensure it falls within the allowed timeframe of **7:00 AM to 6:00 PM**.
3. If the operation is attempted outside these hours, raise an exception with the following message:
"Operations on the ALLOWANCES table are only allowed between 6:00 AM and 6:00 PM."

Deliverable:

- Write a PL/SQL trigger that implements the described functionality.



Solution: Enforcing HR Transaction Restriction

```
CREATE OR REPLACE TRIGGER restrict_allowance_hours
BEFORE INSERT OR UPDATE OR DELETE ON ALLOWANCES
FOR EACH ROW
DECLARE
    -- Declare an exception to handle invalid transaction time
    invalid_time EXCEPTION;
    v_current_hour NUMBER;
BEGIN
    -- Check the current time in 24-hour format
    v_current_hour := TO_NUMBER(TO_CHAR(SYSDATE, 'HH24'));

    -- Check if operation is performed outside working hours (7:00 AM to 6:00 PM)
    IF v_current_hour < 7 OR v_current_hour >= 18 THEN
        RAISE invalid_time;
    END IF;

    -- Additional validations only for INSERT and UPDATE operations
    IF INSERTING OR UPDATING THEN
        -- If allowance amount is invalid (negative), raise an exception
        IF :NEW.ALLOWANCE_AMOUNT < 0 THEN
            RAISE_APPLICATION_ERROR(-20003, 'Allowance amount cannot be negative.');
        END IF;

        -- If the effective date is in the past, raise an exception
        IF :NEW.EFFECTIVE_DATE < SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20004, 'Effective date cannot be in the past.');
        END IF;

        -- If IS_APPLICABLE is not provided, raise an exception
        IF :NEW.IS_APPLICABLE IS NULL THEN
            RAISE_APPLICATION_ERROR(-20005, 'Applicability status (IS_APPLICABLE) is required.');
        END IF;

        -- Check that COMMENTS field is not empty or null
        IF :NEW.COMMENTS IS NULL OR TRIM(:NEW.COMMENTS) IS NULL THEN
            RAISE_APPLICATION_ERROR(-20006, 'Comments are required.');
        END IF;
    END IF;

EXCEPTION
    WHEN invalid_time THEN
        -- Handle invalid time exception with custom message
        RAISE_APPLICATION_ERROR(-20002, 'Operations on the ALLOWANCES table are only allowed between 7:00 AM and 6:00 PM.');
    WHEN OTHERS THEN
        -- Generic exception handler for unexpected errors
        RAISE_APPLICATION_ERROR(-20001, 'An unexpected error occurred: ' || SQLERRM);
END;
```

NEW is a reference to the new values being inserted or updated in the table. Specifically, **:NEW** refers to the record being affected by the INSERT or UPDATE operation in the **ALLOWANCES** table.

Test: Enforcing HR Transaction Restriction Trigger

```
/*
1. Valid Transaction (Within Working Hours):
*/
-- Example: Insert at 10:00 AM with valid data
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (1, 500, SYSDATE + 1, 'Y', 'Annual Bonus');
-- Success: Operation allowed.

/*
2. Invalid Transaction (Outside Working Hours):
*/
-- Example: Insert at 8:00 PM (outside working hours)
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (2, 600, SYSDATE + 1, 'Y', 'Holiday Allowance');
-- Error: "Operations on the ALLOWANCES table are only allowed between 6:00 AM and 6:00 PM."

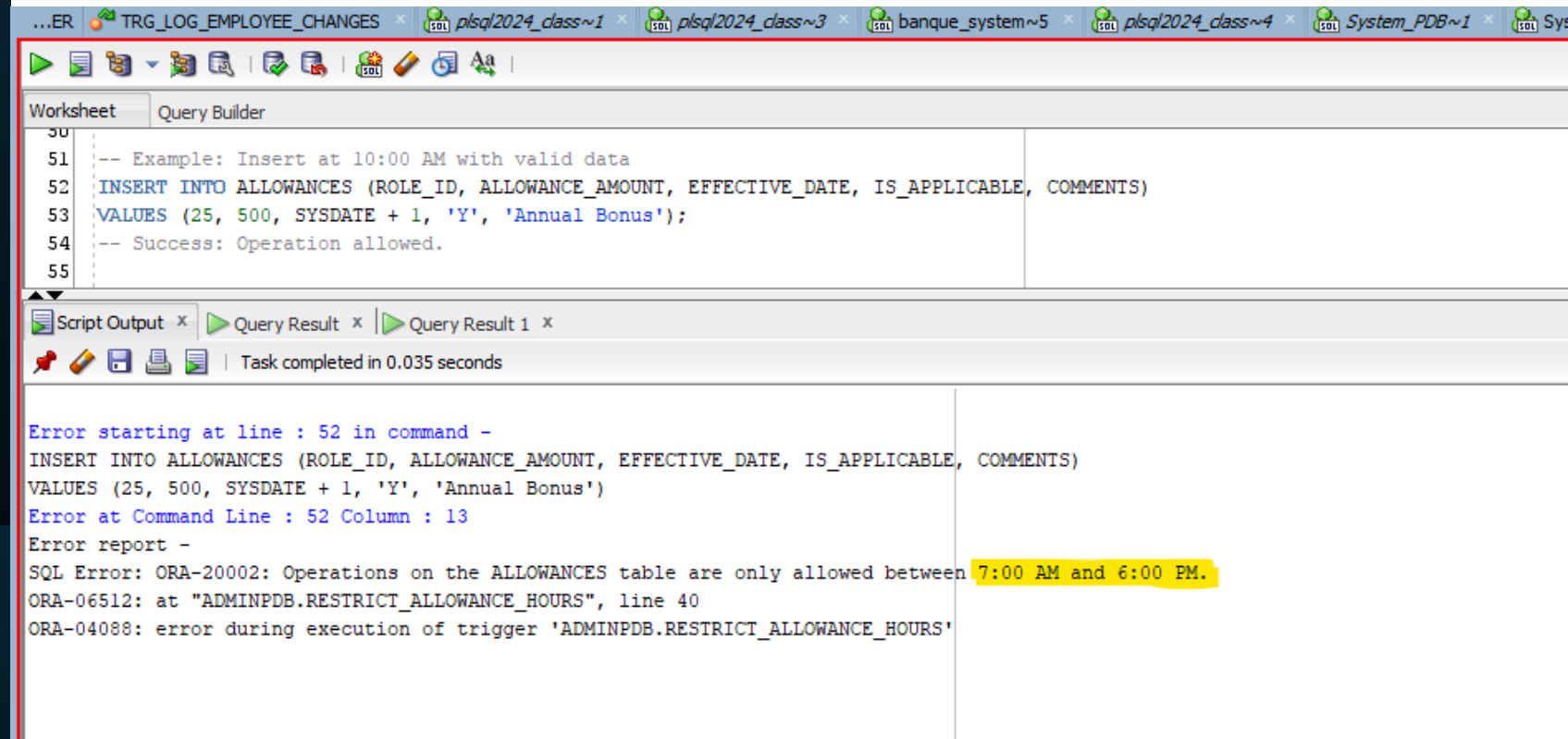
/*
3. Invalid Transaction (Negative Allowance Amount):
*/
-- Example: Insert with a negative allowance amount
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (3, -200, SYSDATE + 1, 'Y', 'Error Test');
-- Error: "Allowance amount cannot be negative."

/*
4. Invalid Transaction (Effective Date in the Past):
*/
-- Example: Insert with an effective date in the past
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (4, 300, SYSDATE - 1, 'Y', 'Retroactive Allowance');
-- Error: "Effective date cannot be in the past."

/*
5. Invalid Transaction (Null Comments):
*/
-- Example: Insert with no comments provided
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (5, 400, SYSDATE + 1, 'Y', NULL);
-- Error: "Comments are required."
```

Test: Enforcing HR Transaction Restriction Trigger

Test out of working hours



The screenshot shows the SQL Developer interface with a worksheet containing an SQL insert statement. The statement is as follows:

```
51 -- Example: Insert at 10:00 AM with valid data
52 INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
53 VALUES (25, 500, SYSDATE + 1, 'Y', 'Annual Bonus');
54 -- Success: Operation allowed.
55
```

Below the worksheet, the 'Script Output' pane shows the execution results. It indicates that the task completed in 0.035 seconds. However, an error occurred starting at line 52 in the command. The error report shows the following messages:

```
Error starting at line : 52 in command -
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (25, 500, SYSDATE + 1, 'Y', 'Annual Bonus')
Error at Command Line : 52 Column : 13
Error report -
SQL Error: ORA-20002: Operations on the ALLOWANCES table are only allowed between 7:00 AM and 6:00 PM.
ORA-06512: at "ADMINPDB.RESTRICT_ALLOWANCE_HOURS", line 40
ORA-04088: error during execution of trigger 'ADMINPDB.RESTRICT_ALLOWANCE_HOURS'
```

The error message 'ORA-20002: Operations on the ALLOWANCES table are only allowed between 7:00 AM and 6:00 PM.' is highlighted in yellow, indicating that the insert operation failed because it was performed outside the allowed working hours.

Scenario-Based Question: Trigger Event, Restricting Bank Transactions Outside Working Hours

Scenario: Enforcing Transaction Timing Policy via PL/SQL Trigger

The Bank of Kigali has introduced a new operational policy aimed at strengthening compliance and enhancing system security. According to this policy, **tellers are prohibited from recording transactions outside official working hours.**

Business Requirements:

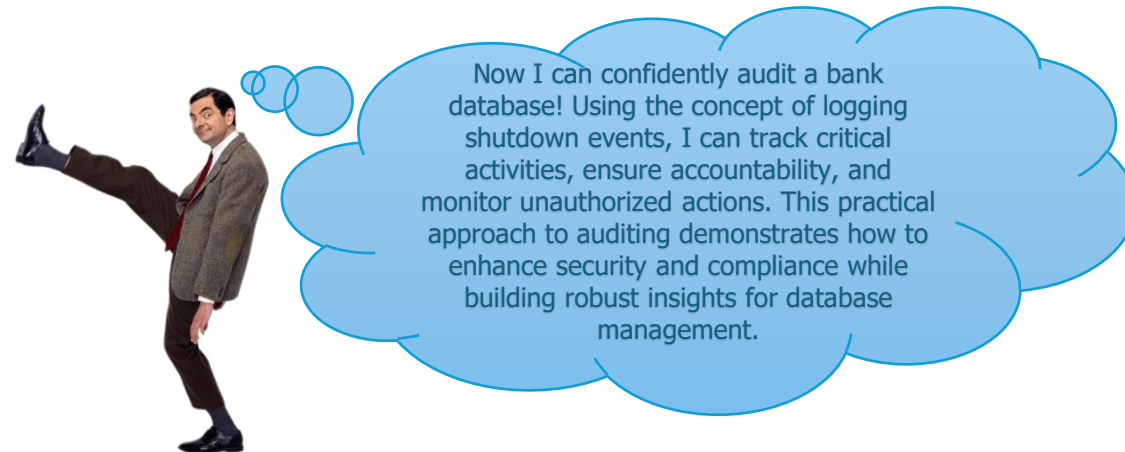
- Transactions may only be recorded **Monday through Friday**, between **9:00 AM and 6:00 PM**.
- Any attempt to record a transaction **outside these hours** or on **weekends (Saturday or Sunday)** must be automatically blocked by the database system.

Task for the Database Developer:

As a database developer, your responsibilities are as follows:

- **Create a PL/SQL Trigger**
 - The trigger must fire **BEFORE INSERT or UPDATE** on the transactions table.
- **Implement Validation Logic**
 - The trigger should **check the current system date and time** using SYSDATE to ensure the transaction is being recorded within the permitted timeframe.
- **Handle Exceptions**
 - If a transaction is attempted **outside the allowed days or hours**, the trigger must:
 - ✓ **Raise an exception with the message:**

"Transactions can only be recorded **between 9:00 AM and 6:00 PM**, Monday to Friday."
This ensures that all transaction records comply with the defined business rules and operational hours.



Solution : Trigger Event - Restricting Bank Transactions

-- Step 1: Create a sample transactions table (if it doesn't already exist)

```
CREATE TABLE transactions (  
    transaction_id NUMBER PRIMARY KEY,  
    teller_id NUMBER,  
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    amount NUMBER  
);
```

-- Step 2: Create the trigger to enforce the policy

```
CREATE OR REPLACE TRIGGER trg_validate_transaction_time  
BEFORE INSERT OR UPDATE ON transactions  
FOR EACH ROW  
BEGIN  
    -- Get the current day of the week and time  
    IF TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('SAT', 'SUN') THEN  
        -- Raise an exception for weekends  
        RAISE_APPLICATION_ERROR(-20001,  
            'Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday.');  
        ELSIF TO_CHAR(SYSDATE, 'HH24MI') < '0900' OR TO_CHAR(SYSDATE, 'HH24MI') > '1800' THEN  
            -- Raise an exception for outside working hours  
            RAISE_APPLICATION_ERROR(-20002,  
                'Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday.');  
        END IF;  
    END;  
/
```

*/**

Testing the Trigger

1. Valid Transaction:

**/*

-- Example of a valid transaction (Monday at 10:00 AM)

```
INSERT INTO transactions (transaction_id, teller_id, amount)  
VALUES (1, 101, 500);
```

Success: Transaction is recorded.

Solution & Test : Trigger Event - Restricting Bank Transactions

```
/*  
Testing the Trigger  
1. Valid Transaction:  
*/  
INSERT INTO transactions (transaction_id, teller_id, amount)  
VALUES (1, 101, 500);
```

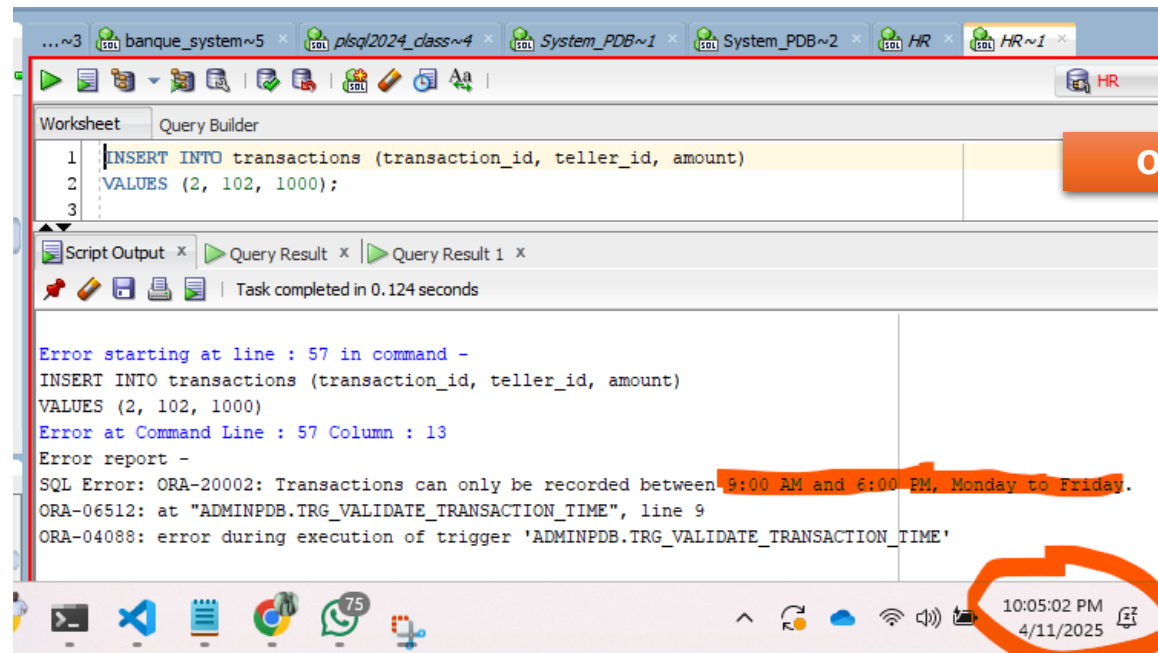
-- Example of a valid transaction (Monday at 10:00 AM)

```
/*  
Invalid Transaction (Weekend): -- Example of a transaction attempted on a Saturday  
*/  
INSERT INTO transactions (transaction_id, teller_id, amount)  
VALUES (2, 102, 1000);
```

-- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."

```
/*  
Invalid Transaction (Outside Working Hours):  
*/  
INSERT INTO transactions (transaction_id, teller_id, amount)  
VALUES (3, 103, 1500);  
-- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."
```

-- Example of a transaction attempted at 8:00 PM on a Monday



Output

Trigger Event: Before Update Trigger to Enforce Salary Protection in Employees Table

Question:

Suppose you are managing an employees table where each employee has a salary that should not be decreased. Write a trigger named prevent_salary_reduction that prevents any update to the salary column if the new salary is lower than the existing salary.

The trigger should meet the following requirements:

1. It should be a **BEFORE UPDATE** trigger, so it checks the salary change before the update is applied.
2. It should raise an error with the message "**Salary reduction is not allowed.**" if an attempt is made to reduce an employee's salary.
3. Use RAISE_APPLICATION_ERROR with a custom error code to handle this restriction.

Provide the full trigger code and explain each part of the code in comments.



```
-- Trigger to prevent salary reduction in the 'employees' table
CREATE OR REPLACE TRIGGER prevent_salary_reduction
-- The trigger will fire before any update on the salary column in the employees table
BEFORE UPDATE OF salary ON employees
FOR EACH ROW -- Specifies that the trigger will execute once for each row being updated
BEGIN
    -- Check if the new salary is less than the old salary
    IF :NEW.salary < :OLD.salary THEN
        -- Raise an application error if the new salary is less than the old salary
        RAISE_APPLICATION_ERROR(-20001, 'Salary reduction is not allowed.');
```

```
--Test my trigger
UPDATE employees
SET salary = 2500000
WHERE employee_id = 5;
```

Test Trigger Event Salary Reduction

```
15
16
17 SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID = 417;
18
```

Script Output x Query Result x											
All Rows Fetched: 1 in 0.002 seconds											
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	ADDRESS	HIRE_DATE	DEPARTMENT_ID	ROLE_ID	SALARY	BONUS
1	417	Emp3178	User4915	emp469luser8733@example.com	07711137723	City 7, Rwanda	10-FEB-24	26	1	100000	10000

Current Salary Output

```
16
17 SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID = 417;
18
19 --Test my trigger
20 UPDATE employees
21 SET salary = 90000
22 WHERE employee_id = 417;
```

Downgrade salary Test Output

Script Output x Query Result x

Task completed in 0.044 seconds

1 row updated.

Error starting at line : 20 in command -
UPDATE employees
SET salary = 90000
WHERE employee_id = 417
Error at Command Line : 20 Column : 8
Error report -
SQL Error: ORA-20001: Salary reduction is not allowed.
ORA-06512: at "ADMINPDB.PREVENT_SALARY_REDUCTION", line 5
ORA-04088: error during execution of trigger 'ADMINPDB.PREVENT_SALARY_REDUCTION'

Creating a Trigger to Log Employee Salary Changes and Operations



BEFORE INSERT OR UPDATE OR DELETE

Suppose you are tasked with creating a trigger that logs salary changes for employees, but only for those whose `ROLE_ID` is not equal to 1 (assuming role 1 is the Manager). The trigger should work for `INSERT`, `UPDATE`, or `DELETE` operations on the `EMPLOYEES` table.

Write a trigger named `log_salary_changes` that fulfills the following requirements:

1. The trigger should be fired **before** an `INSERT`, `UPDATE`, or `DELETE` operation on the `EMPLOYEES` table.
2. It should log the following details for each type of operation:
 - For an `UPDATE` operation: The old and new salary values and the difference between them.
 - For an `INSERT` operation: The name of the new employee and their starting salary.
 - For a `DELETE` operation: The name of the deleted employee and their final salary.
3. The trigger should only log changes for employees who do not have the role of Manager (`ROLE_ID <> 1`).
4. Make use of the `DBMS_OUTPUT.PUT_LINE` procedure to display the logged information.

You should handle all operations appropriately within the trigger, using the `UPDATING`, `INSERTING`, and `DELETING` conditions to check the operation type.

Hint: Use the `:NEW` and `:OLD` references to capture the values of the columns before and after the operations.



Trigger Event: Log Employee Salary Changes and Operations – Solution



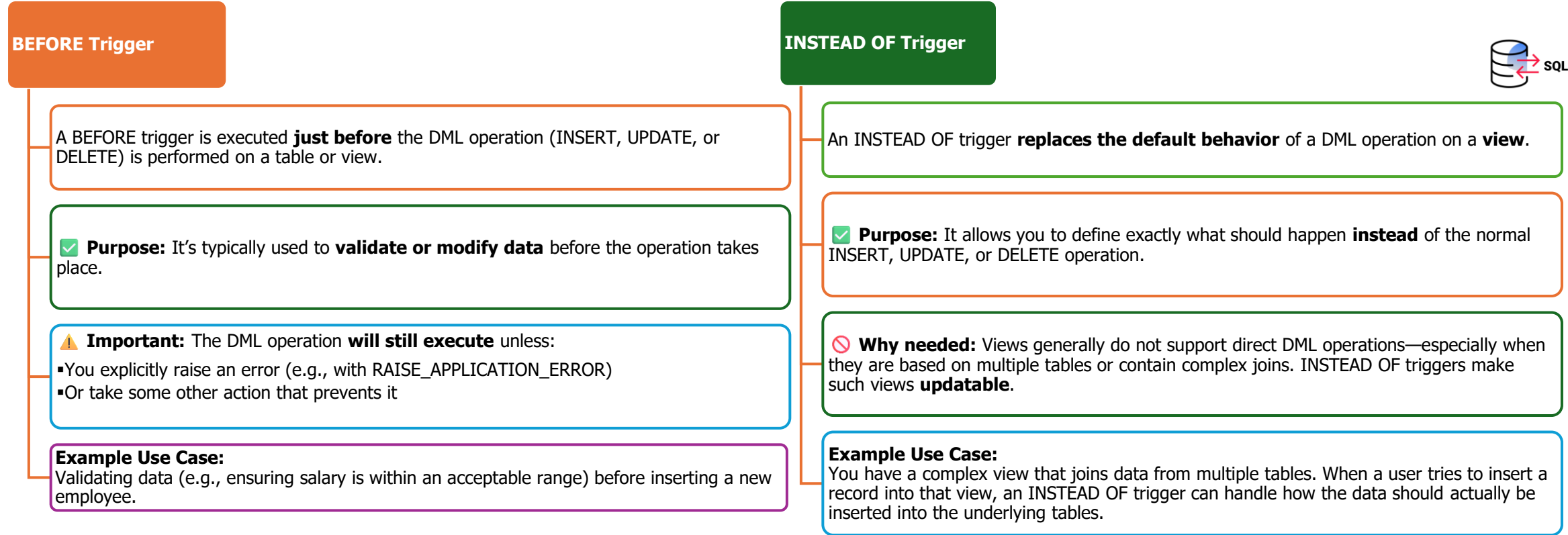
```
CREATE OR REPLACE TRIGGER log_salary_changes
BEFORE INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
WHEN (NEW.ROLE_ID <> 1) -- Assuming '1' is the role ID for the Manager
DECLARE
    sal_diff NUMBER; -- Variable to hold the salary difference
BEGIN
    -- Check if the operation is an update to log salary changes
    IF UPDATING THEN
        sal_diff := :NEW.SALARY - :OLD.SALARY;
        DBMS_OUTPUT.PUT_LINE('Updating Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
        DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
    ELSIF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('Inserting New Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Starting Salary: ' || :NEW.SALARY);
    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Deleting Employee: ' || :OLD.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Final Salary: ' || :OLD.SALARY);
    END IF;
END;
/
```

Hint: Use the :NEW and :OLD references to capture the values of the columns before and after the operations.

```
/*Test Insert Operation*/
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME, LAST_NAME, ROLE_ID, SALARY)
VALUES (1001, 'John', 'Doe', 2, 4000); -- Assuming role ID '2' is for a non-manager
```

```
/* Test Update Operation */
UPDATE EMPLOYEES
SET SALARY = 4500
WHERE EMPLOYEE_ID = 1001;
```

Difference Between BEFORE and INSTEAD OF Triggers



Summary Table

Trigger Type	When It Executes	Used On	Default Action
BEFORE	Before the DML runs	Tables	Operation proceeds unless stopped
INSTEAD OF	Replaces the DML logic	Views	Custom logic fully replaces default behavior

INSTEAD OF Trigger

```
/*
2. Create the INSTEAD OF Trigger:
*/
CREATE OR REPLACE TRIGGER log_salary_changes
INSTEAD OF INSERT OR UPDATE OR DELETE ON EMPLOYEES_VIEW
FOR EACH ROW
DECLARE
    sal_diff NUMBER;
BEGIN
    -- Handling INSERT operation
    IF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('Inserting New Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Starting Salary: ' || :NEW.SALARY);
        -- Perform actual insert action
        INSERT INTO EMPLOYEES (EMPLOYEE_ID, LAST_NAME, SALARY)
        VALUES (:NEW.EMPLOYEE_ID, :NEW.LAST_NAME, :NEW.SALARY);

    ELSIF UPDATING THEN
        sal_diff := :NEW.SALARY - :OLD.SALARY;
        DBMS_OUTPUT.PUT_LINE('Updating Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
        DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
        -- Perform actual update action
        UPDATE EMPLOYEES
        SET SALARY = :NEW.SALARY
        WHERE EMPLOYEE_ID = :OLD.EMPLOYEE_ID;

    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Deleting Employee: ' || :OLD.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Final Salary: ' || :OLD.SALARY);
        -- Perform actual delete action
        DELETE FROM EMPLOYEES WHERE EMPLOYEE_ID = :OLD.EMPLOYEE_ID;
    END IF;
END;
/
```

```
/*1. Create the View (if not already created):*/
```

```
CREATE OR REPLACE VIEW EMPLOYEES_VIEW AS
SELECT EMPLOYEE_ID, LAST_NAME, SALARY
FROM EMPLOYEES;
```

Instead Of Event: Log Employee Salary Changes and Operations – Solution

View-Based Operations

- The trigger is defined **on a view** named EMPLOYEES_VIEW, not directly on the base table EMPLOYEES.
- This approach allows controlled manipulation of data presented through the view, especially when the view is complex or combines multiple tables.

INSTEAD OF Trigger

- The **INSTEAD OF trigger** intercepts INSERT, UPDATE, and DELETE operations **on the view**.
- It allows you to define **custom logic** that replaces the default behavior—like logging, validating, or rerouting the changes to the underlying table (EMPLOYEES in this case).

Use of DBMS_OUTPUT

- The DBMS_OUTPUT.PUT_LINE procedure is used for **logging** purposes.
- It prints useful information (like salary differences) to help with **debugging or auditing**.

Custom Actions

- The trigger can include **custom logic**, such as logging operations to a separate audit table, enforcing rules, or notifying users.
- After logging or validation, the **actual DML operation is performed manually** on the EMPLOYEES table from within the trigger body.

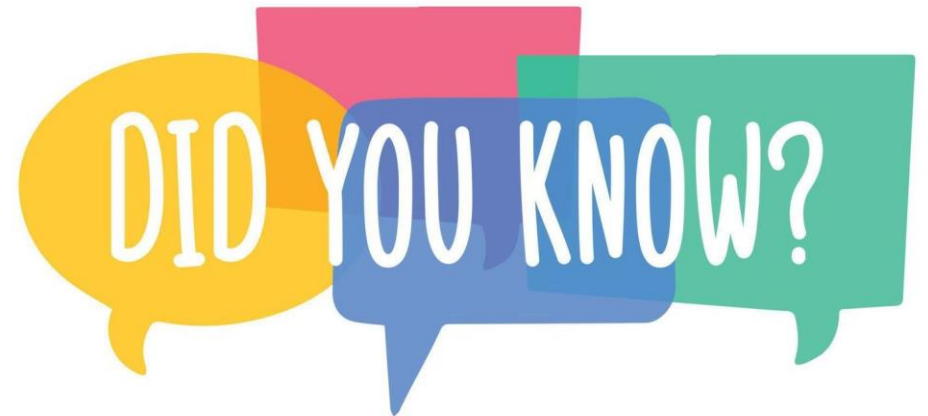
Important Considerations

1. No Direct Changes to the View:

- INSTEAD OF triggers don't modify the view's data directly.
- They handle the logic needed to **manipulate the underlying base table(s)** instead.

2. Only for Views:

- INSTEAD OF triggers are **only applicable to views**.
- They are especially useful for making **non-updatable views behave like updatable ones** by defining what should happen during DML operations.



Oracle Trigger Execution Order with FOLLOWS Clause

In Oracle, you can define multiple triggers for the same timing point (*e.g., BEFORE INSERT, AFTER UPDATE*). However, before **Oracle 11g**, their execution order was not guaranteed, which could cause inconsistent behavior in complex systems. Starting with Oracle 11g, the FOLLOWS clause allows you to explicitly control the execution order of such triggers. This ensures predictable and consistent behavior when multiple triggers are defined for the same timing point.

Example: The following demonstrates how to create a table and two **BEFORE INSERT** triggers, using **FOLLOWS** to define their execution order.

- Table Created
- Trigger 1 created

```
-- Create a table to test trigger execution
CREATE TABLE trigger_follows_test (
  id      NUMBER,      -- Unique identifier for each row
  description VARCHAR2(50) -- Description for each row
);

-- Create the first trigger
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_1
BEFORE INSERT ON trigger_follows_test -- Trigger fires before an insert
FOR EACH ROW                          -- Fires for each row inserted
BEGIN
  -- Print message indicating this trigger was executed
  DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_1 - Executed');
END;
/
```

- Trigger 2 created

```
-- Create the second trigger with FOLLOWS clause
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_2
BEFORE INSERT ON trigger_follows_test -- Trigger fires before an insert
FOR EACH ROW                          -- Fires for each row inserted
FOLLOWS trigger_follows_test_trg_1    -- Specifies that this trigger executes after
trigger_1
BEGIN
  -- Print message indicating this trigger was executed
  DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_2 - Executed');
END;
/
```

Trigger Execution Order with - Test Triggers

```
SET SERVEROUTPUT ON;
```

```
-- Test the triggers by inserting a record
```

```
INSERT INTO trigger_follows_test (id, description)  
VALUES (1, 'Testing trigger execution order');
```

```
-- Verify the inserted record
```

```
SELECT * FROM trigger_follows_test;
```

```
-- Create a third trigger to demonstrate multiple FOLLOWS relationships
```

```
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_3
```

```
BEFORE INSERT ON trigger_follows_test
```

```
FOR EACH ROW
```

```
FOLLOWS trigger_follows_test_trg_2
```

```
BEGIN
```

```
DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_3 - Executed');
```

```
END;
```

```
/
```

```
-- Test with the third trigger
```

```
INSERT INTO trigger_follows_test (id, description)  
VALUES (2, 'Testing with three triggers');
```

- Insert data in `table trigger_follows_test`
- Verify our records from the table
- Trigger 3 created
- Insert again data in `table trigger_follows_test`

Clean up (optional)
DROP TABLE trigger_follows_test CASCADE
CONSTRAINTS;

```
41 FOR EACH ROW  
42 FOLLOWS trigger_follows_test_trg_2  
43 BEGIN  
44     DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_3 - Executed');  
45 END;  
46 /  
47  
48 -- Test with the third trigger  
49 INSERT INTO trigger_follows_test (id, description)  
50 VALUES (2, 'Testing with three triggers');  
51  
52
```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x Query Result 4 x Qu

Task completed in 0.029 seconds

Trigger TRIGGER_FOLLOWS_TEST_TRG_3 compiled

TRIGGER_FOLLOWS_TEST_TRG_1 - Executed
TRIGGER_FOLLOWS_TEST_TRG_2 - Executed
TRIGGER_FOLLOWS_TEST_TRG_3 - Executed

1 row inserted.

Output

Oracle Trigger Execution Order with FOLLOWS Clause - Summary

- **Without the FOLLOWS clause:** The execution order of triggers at the same timing point is unpredictable.
- **With the FOLLOWS clause:** You can explicitly define the execution order, ensuring consistent and predictable behavior.

The FOLLOWS clause is especially useful in complex systems where triggers rely on the output of one another.



Compound Triggers in Oracle PL/SQL

A **compound trigger** in PL/SQL (introduced in **Oracle 11g**) is a powerful feature that allows you to define **multiple trigger timing points**—such as:

- BEFORE STATEMENT
- BEFORE EACH ROW
- AFTER EACH ROW
- AFTER STATEMENT

—**within a single trigger block** on the same table.

This unified structure helps you manage complex business logic more efficiently and **avoids common issues like the mutating table error**, which can occur with regular row-level triggers.

Share Data Across Timing Points

Compound triggers let you share variables and temporary data across different timing sections, enabling consistent and simplified processing.

Improve Performance with Bulk Operations

Instead of processing each row individually, you can collect data during row-level events and process it in bulk in the AFTER STATEMENT section—boosting performance.

Avoid Mutating Table Errors

By deferring certain operations and using temporary storage, compound triggers help prevent **mutating table errors**, which occur when a trigger accesses or modifies the table that caused it to fire.

Restrictions of Compound Triggers



DML-Only:

Compound triggers can only be defined for INSERT, UPDATE, or DELETE operations on **tables or views**.



Compound Structure Required:

The trigger **body must follow** the compound trigger syntax, defining sections like BEFORE STATEMENT, BEFORE EACH ROW, etc.



No Initialization Block:

You **cannot include an initialization block**. All variables must be initialized in the declaration or within timing sections.



:OLD and :NEW Scope Limited:

These pseudorecords are **only accessible** in BEFORE EACH ROW and AFTER EACH ROW sections—not in DECLARE, BEFORE STATEMENT, or AFTER STATEMENT.



Unpredictable Firing Order:

If multiple compound triggers exist on the same table, their firing order is **not guaranteed** unless controlled using FOLLOWS or PRECEDES.



Best Practices for Writing Compound Triggers

1

Use Shared Variables:
Declare variables in the trigger body to share data across timing points.

2

Apply Bulk Processing:
Collect data during row-level sections and process it in bulk in the AFTER STATEMENT section for **better performance**.

3

Keep Logic Modular:
Distribute logic cleanly across timing points. Avoid overloading any single section with complex code.

4

Prevent Mutating Table Errors:
Temporarily store row-level data and act on it in AFTER STATEMENT to **avoid accessing the table while it's being modified**.



Advantages of Compound Triggers

Simplified Maintenance:
Combines logic across multiple timing points into **one unified trigger**, reducing code duplication.

Performance Boost:
Improves performance with **bulk processing**, especially in high-volume operations.

Error Prevention:
Helps avoid **mutating table errors** and other common trigger pitfalls.

Enhanced Flexibility:
Allows logic to execute at **different stages** of the DML lifecycle.



Compound Triggers Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
FOR INSERT OR UPDATE OR DELETE ON object_name
[REFERENCING OLD AS old NEW AS new]
COMPOUND TRIGGER
```

```
-- Declaration Section (optional)
-- Shared between all timing points
variable_name datatype;
```

```
-- Before Statement Section (optional)
BEFORE STATEMENT IS
BEGIN
-- Code executed once before the statement
EXCEPTION
WHEN OTHERS THEN
-- Error handling code (optional)
END BEFORE STATEMENT;
```

```
-- Before Each Row Section (optional)
BEFORE EACH ROW IS
BEGIN
-- Code executed before each row operation
EXCEPTION
WHEN OTHERS THEN
-- Error handling code (optional)
END BEFORE EACH ROW;
```

```
-- After Each Row Section (optional)
AFTER EACH ROW IS
BEGIN
-- Code executed after each row operation
EXCEPTION
WHEN OTHERS THEN
-- Error handling code (optional)
END AFTER EACH ROW;
```

```
-- After Statement Section (optional)
AFTER STATEMENT IS
BEGIN
-- Code executed once after the statement
EXCEPTION
WHEN OTHERS THEN
-- Error handling code (optional)
END AFTER STATEMENT;
```

```
END trigger_name;
```

Optional Section

Syntax:

```
CREATE [ OR REPLACE ] TRIGGER <trigger_name>
FOR
[ INSERT | UPDATE | DELETE ..... ]
ON <name of underlying object>
<Declarative part>
BEFORE STATEMENT IS
BEGIN
    <Execution part>;
END BEFORE STATEMENT;

BEFORE EACH ROW IS
BEGIN
    <Execution part>;
END EACH ROW;

AFTER EACH ROW IS
BEGIN
    <Execution part>;
END AFTER EACH ROW;

AFTER STATEMENT IS
BEGIN
    <Execution part>;
END AFTER STATEMENT;
END;
```

BEFORE-
STATEMENT Level

BEFORE-
ROW Level

AFTER-ROW
Level

AFTER-
STATEMENT Level

Source Image: <https://www.guru99.com/triggers-pl-sql.html>

Syntax Explanation:

- This syntax shows the creation of '**COMPOUND**' trigger.
- Declarative section is common for all the execution block in the trigger body.
- These 4 timing blocks can be in any sequence. It is not mandatory to have all these 4 timing blocks. We can create a COMPOUND trigger only for the timings which are required.



"The function of education is to teach one
to think intensively and to think critically"
— Martin Luther King Jr.

Example of Compound (e.g. Student Table)

```
CREATE OR REPLACE TRIGGER trg_student_changes  
FOR INSERT OR UPDATE OR DELETE ON students  
COMPOUND TRIGGER
```

```
-- Declaration Section: Shared between all timing points  
row_counter INTEGER := 0;
```

```
-- Before Statement Section
```

```
BEFORE STATEMENT IS  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Starting DML operation on STUDENTS table...');  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error in BEFORE STATEMENT: ' || SQLERRM);  
END BEFORE STATEMENT;
```

```
-- Before Each Row Section
```

```
BEFORE EACH ROW IS  
BEGIN  
    -- Validate GPA range  
    IF INSERTING OR UPDATING THEN  
        IF :NEW.gpa < 0 OR :NEW.gpa > 5 THEN  
            RAISE_APPLICATION_ERROR(-20001, 'GPA must be between 0 and 5');  
        END IF;
```

```
-- Ensure student name is not NULL
```

```
    IF :NEW.student_name IS NULL THEN  
        RAISE_APPLICATION_ERROR(-20002, 'Student name cannot be NULL');  
    END IF;  
END IF;  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error in BEFORE EACH ROW: ' || SQLERRM);  
END BEFORE EACH ROW;
```

```
-- After Each Row Section
```

```
AFTER EACH ROW IS  
BEGIN  
    row_counter := row_counter + 1;  
  
    IF INSERTING THEN  
        DBMS_OUTPUT.PUT_LINE('Inserted: ' || :NEW.student_name || ',  
GPA: ' || :NEW.gpa);  
    ELSIF UPDATING THEN  
        DBMS_OUTPUT.PUT_LINE('Updated: From GPA ' || :OLD.gpa || ' to '  
|| :NEW.gpa);  
    ELSIF DELETING THEN  
        DBMS_OUTPUT.PUT_LINE('Deleted: ' || :OLD.student_name);  
    END IF;  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error in AFTER EACH ROW: ' ||  
SQLERRM);  
END AFTER EACH ROW;
```

```
-- After Statement Section
```

```
AFTER STATEMENT IS  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Total rows processed: ' || row_counter);  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error in AFTER STATEMENT: ' ||  
SQLERRM);  
END AFTER STATEMENT;
```

```
END trg_student_changes;  
/
```

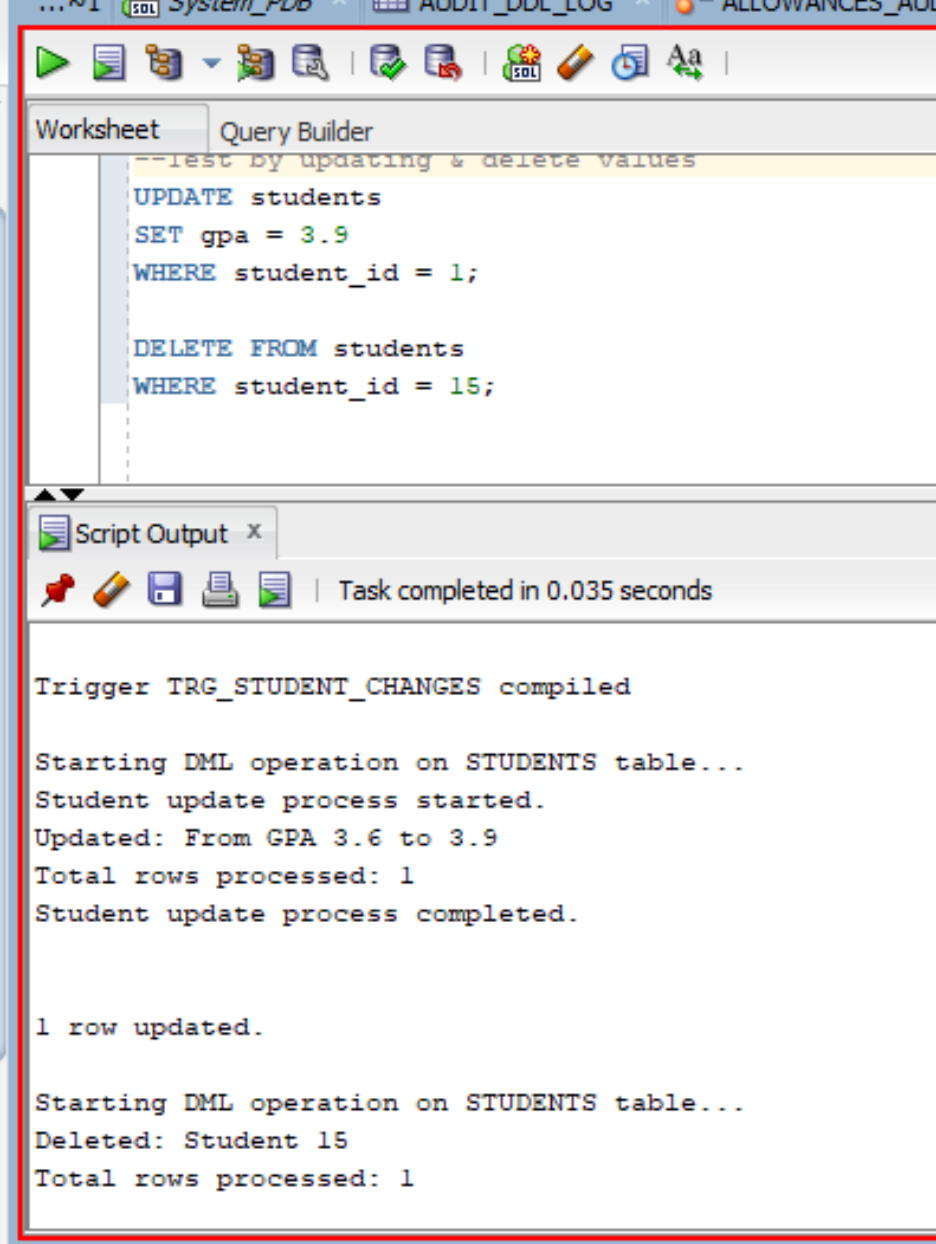
Example of Compound (e.g. Student Table) → Test

Test by updating

```
UPDATE students  
SET gpa = 3.9  
WHERE student_id = 1;
```

Test by delete

```
DELETE students  
WHERE studentId = 15;
```



Example: Auditing Data Changes with a Compound Trigger

How can we use a compound trigger to track and log salary changes (inserts, updates, and deletes) on the EMPLOYEES table into an AUDIT_EMP table, while ensuring performance and avoiding mutating table errors?

→ Purpose

The compound_salary_audit trigger tracks salary changes on the EMPLOYEES table and stores them in a separate salary_audit table using bulk processing for better performance and to avoid mutating table errors. write this clearly as a question

Objects Involved

-- Audit Table

```
CREATE TABLE salary_audit (  
  employee_id NUMBER,  
  old_salary NUMBER,  
  new_salary NUMBER,  
  change_date DATE  
);
```

Object Type and Nested Table

```
-- Create an object type to represent a single employee's salary change  
CREATE OR REPLACE TYPE employee_salary_obj AS OBJECT (  
  employee_id NUMBER, -- Employee's ID  
  old_salary NUMBER, -- Salary before the update  
  new_salary NUMBER -- Salary after the update  
);  
/  
  
-- Create a nested table type to store multiple employee salary change records  
CREATE OR REPLACE TYPE employee_salary_tab IS TABLE OF employee_salary_obj;  
/
```

Audit Salary with Shared Object Type - Trigger Logic

```
-- Create a compound trigger to audit salary updates in the EMPLOYEES table
CREATE OR REPLACE TRIGGER compound_salary_audit
FOR UPDATE OF SALARY ON EMPLOYEES
COMPOUND TRIGGER

    -- Declare a nested table variable to collect salary changes during the statement
    salary_changes employee_salary_tab := employee_salary_tab();

    -- BEFORE EACH ROW section: captures the salary change before each row is updated
    BEFORE EACH ROW IS
    BEGIN
        -- Extend the nested table to add a new entry
        salary_changes.EXTEND;

        -- Store the current row's employee ID, old salary, and new salary
        salary_changes(salary_changes.COUNT) := employee_salary_obj(
            :OLD.EMPLOYEE_ID,
            :OLD.SALARY,
            :NEW.SALARY
        );
    END BEFORE EACH ROW;

    -- AFTER STATEMENT section: runs once after all row updates are done
    AFTER STATEMENT IS
    BEGIN
        -- Loop through the collected changes and insert them into the audit table
        FOR i IN 1 .. salary_changes.COUNT LOOP
            INSERT INTO salary_audit (
                employee_id,
                old_salary,
                new_salary,
                change_date
            ) VALUES (
                salary_changes(i).employee_id,
                salary_changes(i).old_salary,
                salary_changes(i).new_salary,
                SYSDATE
            );
        END LOOP;

        -- Clear the nested table after inserting audit records
        salary_changes.DELETE;
    END AFTER STATEMENT;

END compound_salary_audit;
/
```

```
-- Update salaries by 10% for employees in department 3
UPDATE EMPLOYEES
SET SALARY = SALARY * 1.10
WHERE DEPARTMENT_ID = 3;

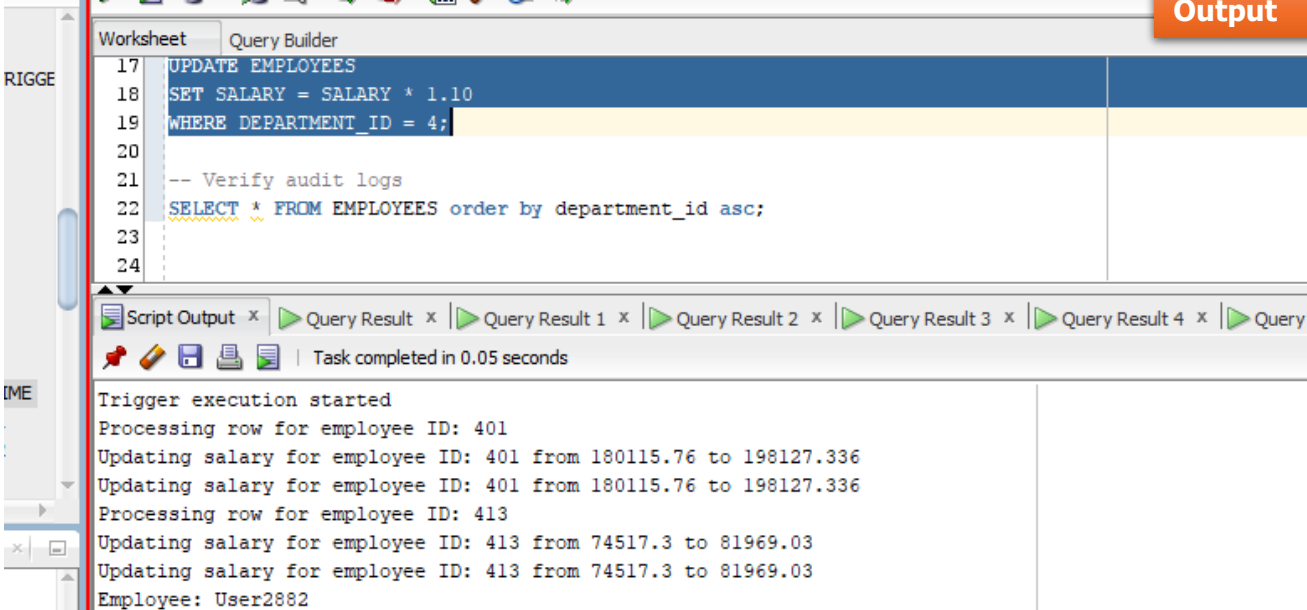
-- Verify audit logs
SELECT * FROM salary_audit;
```

Audit Salary with Shared Object Type - Test Case

Key Benefits

- **Performance:** Uses bulk logging instead of row-by-row INSERTs.
- **Prevents Mutating Table Errors:** Defers DML to AFTER STATEMENT.
- **Structured Data:** Uses object types and nested tables for clean handling.

Output

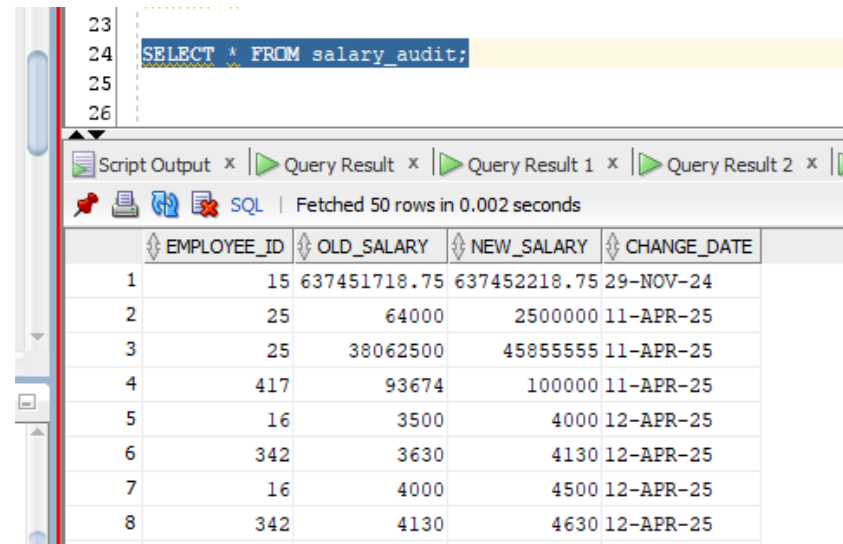


```
17 UPDATE EMPLOYEES
18 SET SALARY = SALARY * 1.10
19 WHERE DEPARTMENT_ID = 4;
20
21 -- Verify audit logs
22 SELECT * FROM EMPLOYEES order by department_id asc;
23
24
```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x Query Result 4 x Query

Task completed in 0.05 seconds

Trigger execution started
Processing row for employee ID: 401
Updating salary for employee ID: 401 from 180115.76 to 198127.336
Updating salary for employee ID: 401 from 180115.76 to 198127.336
Processing row for employee ID: 413
Updating salary for employee ID: 413 from 74517.3 to 81969.03
Updating salary for employee ID: 413 from 74517.3 to 81969.03
Employee: User2882



```
23
24 SELECT * FROM salary_audit;
25
26
```

Script Output x Query Result x Query Result 1 x Query Result 2 x

SQL | Fetched 50 rows in 0.002 seconds

	EMPLOYEE_ID	OLD_SALARY	NEW_SALARY	CHANGE_DATE
1	15	637451718.75	637452218.75	29-NOV-24
2	25	64000	2500000	11-APR-25
3	25	38062500	45855555	11-APR-25
4	417	93674	100000	11-APR-25
5	16	3500	4000	12-APR-25
6	342	3630	4130	12-APR-25
7	16	4000	4500	12-APR-25
8	342	4130	4630	12-APR-25

Output

Question: Compound of Maintaining Employee Salary Records Automatically

Scenario:

Your company maintains two tables:

1. **EMPLOYEES:** Stores EMPLOYEE_ID, SALARY, and BONUS.
2. **EMPLOYEE_SALARIES:** Tracks employee salary and bonus details for payroll.

Requirement:

Whenever an employee is added or their SALARY or BONUS is updated in the EMPLOYEES table, ensure that the EMPLOYEE_SALARIES table is updated to reflect the changes. Use a **compound trigger** to manage this logic efficiently. If the employee does not exist in EMPLOYEE_SALARIES, insert a new record.

Database Schema:

EMPLOYEES:

- EMPLOYEE_ID (Primary Key)
- SALARY
- BONUS

EMPLOYEE_SALARIES:

- EMPLOYEE_ID (Primary Key)
- SALARY
- BONUS



Tasks:

1. Write a Compound Trigger:

- Use a compound trigger to ensure the following:
 - During the **statement-level operation**, initialize necessary structures (if needed).
 - At the **row-level operation**, update or insert into the EMPLOYEE_SALARIES table as required.
 - Log any errors explicitly.
- Handle duplicate records gracefully to avoid conflicts in the EMPLOYEE_SALARIES table.

2. Add Exception Handling:

- Explicitly handle exceptions for UPDATE or INSERT operations.
- Log meaningful error messages for failures (e.g., primary key violations).

3. Test the Trigger:

- Insert a new employee into the EMPLOYEES table and verify changes in EMPLOYEE_SALARIES.
- Update an existing employee's SALARY or BONUS in EMPLOYEES and ensure the changes are reflected in EMPLOYEE_SALARIES.

Validation Steps:

- Insert a new employee into EMPLOYEES and check the insertion into EMPLOYEE_SALARIES.
- Update an employee's salary or bonus and verify the update in EMPLOYEE_SALARIES.
- Test duplicate handling by attempting to insert an already existing record in EMPLOYEE_SALARIES.

Scenario-Based Question: Auditing Student Data with a Compound Trigger at AUCA

Problem Statement: Student Data Auditing with Compound Triggers

You are the database administrator for **Adventist University of Central Africa (AUCA)**. Your objective is to **track and audit changes** made to student records using a **compound trigger**. This ensures that any updates to the student data are automatically logged in an audit table for accountability and historical tracking.

Task Overview

Follow the steps below to design and implement the auditing system:

Create the Required Tables

- **students Table:**
This table will store student records such as student_id, name, department, etc.
- **students_audit Table:**
This table will log updates made to the students table. Each log entry should capture:
 - student_id
 - action_type (e.g., 'UPDATE')
 - change_date (timestamp of the update)

2. Insert Sample Data

- Populate the students table with several sample records for testing purposes.

3. Define a Compound Trigger

- Create a **compound trigger** on the students table that:
 - Detects **UPDATE** operations.
 - Uses a **collection (nested table)** to store student changes during the row-level operation.
 - Performs **bulk logging** of these changes in the AFTER STATEMENT section by inserting entries into the students_audit table.
 - Stores relevant data such as student_id, action_type, and change_date.

4. Test the Implementation

- Execute **UPDATE** statements on the students table.
- Verify that each change is **accurately logged** in the students_audit table.



Auditing Student Data with a Compound Optional

*/*1. Create Student Table */*

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(100),  
    major VARCHAR2(50),  
    gpa NUMBER(3, 2)  
);
```

*/*2. Student audit table */*

```
CREATE TABLE students_audit (  
    audit_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    student_id NUMBER,  
    action VARCHAR2(50),  
    change_date DATE  
);
```

*/**

Step 2: Insert Sample Data

Populate the students table with sample data:

**/*

```
BEGIN  
    FOR i IN 1..1000 LOOP  
        INSERT INTO students (student_id, student_name, major, gpa)  
        VALUES (  
            i,  
            'Student ' || i,  
            CASE MOD(i, 4)  
                WHEN 0 THEN 'Computer Science'  
                WHEN 1 THEN 'Engineering'  
                WHEN 2 THEN 'Business'  
                ELSE 'Arts'  
            END,  
            2.5 + MOD(i, 5) * 0.1  
        );  
    END LOOP;  
END;
```

-- Display data in students_audit;

```
SELECT * FROM students_audit;
```

OPTIONAL

Note:

In this setup, we are generating **randomized sample data** for the students table using a loop. Each student is assigned:

- A unique student_id from 1 to 1000,
- A name in the format "Student 1", "Student 2", ..., "Student 1000",
- A major chosen cyclically from **Computer Science, Engineering, Business, and Arts**,
- A GPA ranging from **2.5 to 2.9**, calculated using a simple formula to add variation.

This approach is used to simulate realistic and diverse student data for testing the audit logging system.

Solution: Compound Trigger to Track Updates on the students Table

-- Create the compound trigger

```
CREATE OR REPLACE TRIGGER compound_students_audit
FOR UPDATE ON students
COMPOUND TRIGGER
```

-- Declaration section: define the collection to hold audit info

```
student_changes student_audit_tab := student_audit_tab();
```

```
BEFORE EACH ROW IS
BEGIN
```

-- Store only if there's a change in data

```
IF :OLD.gpa != :NEW.gpa THEN
    student_changes.EXTEND;
    student_changes(student_changes.COUNT) := student_audit_obj(:OLD.student_id);
END IF;
END BEFORE EACH ROW;
```

```
AFTER STATEMENT IS
BEGIN
```

-- Bulk insert audit records for changed students

```
FOR i IN 1 .. student_changes.COUNT LOOP
    INSERT INTO students_audit (
        student_id,
        action,
        change_date
    ) VALUES (
        student_changes(i).student_id,
        'UPDATE',
        SYSDATE
    );
END LOOP;
```

-- Clear the collection

```
student_changes.DELETE;
END AFTER STATEMENT;
```

```
END compound_students_audit;
/
```

-- Step 4: Create necessary types for bulk processing

```
CREATE OR REPLACE TYPE student_audit_obj AS OBJECT (
    student_id NUMBER
);
/
CREATE OR REPLACE TYPE student_audit_tab IS TABLE OF student_audit_obj;
/
```

Solution: Compound Trigger to Track Updates Test

```
48
49 UPDATE students SET gpa = gpa + 0.5 WHERE gpa < 4.0
50
```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x

Task completed in 0.317 seconds

Table STUDENTS_AUDIT created.

Student update process started.

Student update process completed.

1,000 rows updated.

Output

-- Step 1: Update GPAs for all students whose GPA is currently below 4.0
-- This will trigger the compound trigger to log each update into the audit table

```
UPDATE students
SET gpa = gpa + 0.5
WHERE gpa < 4.0;
```

```
1 SELECT COUNT(*) FROM students_audit;
2
3
4 SELECT * FROM students_audit WHERE ROWNUM <= 10;
```

Script Output x Query Result x Query Result 1 x Query Result 2 x

SQL | All Rows Fetched: 10 in 0.005 seconds

	AUDIT_ID	STUDENT_ID	ACTION	CHANGE_DATE
1	526	932	UPDATE	12-APR-25
2	527	933	UPDATE	12-APR-25
3	528	934	UPDATE	12-APR-25
4	529	935	UPDATE	12-APR-25
5	530	936	UPDATE	12-APR-25
6	531	937	UPDATE	12-APR-25
7	532	938	UPDATE	12-APR-25
8	533	939	UPDATE	12-APR-25
9	534	940	UPDATE	12-APR-25
10	535	941	UPDATE	12-APR-25

Output

-- Step 2: Check the total number of records inserted into the students_audit table
-- This should match the number of rows updated in the students table

```
SELECT COUNT(*) FROM students_audit;
```

-- Step 3: Retrieve and review the first 10 audit records
-- Useful for verifying that the audit entries were correctly logged

```
SELECT * FROM students_audit
WHERE ROWNUM <= 10;
```

Scenario Question: Managing Allowances Based on Attendance

Here's a scenario that incorporates the creation and use of a compound trigger to manage related data across the ALLOWANCES and ATTENDANCE tables. The trigger ensures that when an employee's attendance record indicates absence, an allowance record is automatically marked as not applicable (IS_APPLICABLE = 'N').

Scenario: Managing Allowances Based on Attendance

Objective:

Ensure that allowances are correctly updated based on employee attendance records. If an employee is marked absent on a particular date, all allowance entries linked to their role and effective on or before that date are marked as not applicable (IS_APPLICABLE = 'N').

Steps

1. Tables Involved:

- ALLOWANCES: Tracks allowances assigned to roles.
- ATTENDANCE: Records daily attendance for employees.

2. Business Rule:

- If an employee is marked as absent (STATUS = 'Absent') on a specific date:
 - All allowances tied to the employee's role (ROLE_ID) and effective on or before the absence date (EFFECTIVE_DATE) must have their IS_APPLICABLE field updated to 'N'.

3. Technical Requirements:

- Use a **compound trigger** to handle the update in a transaction-safe manner.
- Use an **object type** to manage allowances updates as part of the trigger logic.



Scenario Question: PL/SQL Compound Trigger Question: Instagram System

Scenario:

You're working on the backend of an Instagram-like system where users can post pictures, like, comment, and follow other users. Your task is to implement **compound triggers** to manage the operations within the system. A **compound trigger** allows you to group multiple trigger actions (such as INSERT, UPDATE, and DELETE) into a single trigger, which can help to avoid potential issues with firing multiple triggers for the same event.

Requirements:

1. Create a Compound Trigger to Track New Posts:

- Every time a user posts a new picture, you need to automatically record the date and time of the post in a posts_audit table. This table has the following columns: post_id, user_id, post_date.

Question:

- Write a compound trigger that will:
 - Insert a record into the posts_audit table when a new post is added to the posts table.

2. Create a Compound Trigger for Comments on Posts:

- When a new comment is added to a post, you need to update the comments_count column in the posts table to reflect the new number of comments on that post.

Question:

- Write a compound trigger that will:
 - Update the comments_count in the posts table whenever a new comment is added to a post in the comments table.

3. Create a Compound Trigger for User Follows:

- Every time a user follows another user, a notification should be sent to the followed user by inserting a new record into the notifications table. This table has the following columns: notification_id, user_id, follower_id, notification_message, and notification_date.

Question:

- Write a compound trigger that will:
 - Insert a record into the notifications table whenever a user follows another user.

4. Create a Compound Trigger to Prevent Multiple Posts in a Short Timeframe:

- If a user tries to post more than 5 times within 1 minute, the system should prevent the post from being added and raise an error message.

Question:

- Write a compound trigger that will:
 - Restrict a user from posting more than 5 times within a minute and raise an appropriate error if the condition is met.



Scenario Question: PL/SQL Compound Trigger Question: Instagram System

Evaluation Criteria:

- **Use of Compound Triggers:** Understanding how to use the AFTER STATEMENT, AFTER EACH ROW, and BEFORE STATEMENT sections effectively within a compound trigger.
- **Trigger Logic:** Correct handling of multiple actions (insert, update, delete) within a single trigger body.
- **Error Handling:** Proper error handling and raising exceptions when necessary (e.g., for the restriction on multiple posts).
- **Table Relationships:** Correct understanding of the relationships between tables (e.g., posts, comments, notifications).
- **Performance Considerations:** Efficient use of compound triggers to manage multiple actions at once, avoiding redundant or unnecessary triggers.

Bonus Question:

- How would you modify the compound trigger to include a logging mechanism that tracks the status of the post (whether it's public or private) every time a new post is added to the system?



Recap: PLSQL - Trigger

1. DML Triggers (Data Manipulation Language)

These triggers are fired **in response to DML operations** on tables or views:

- INSERT
- UPDATE
- DELETE

They can be defined as **BEFORE**, **AFTER**, or **INSTEAD OF** (on views).

2. INSTEAD OF Triggers

- Specifically designed **for views**.
- Allow performing INSERT, UPDATE, or DELETE on views that normally don't support direct DML operations.
- Executed **instead of** the DML operation on the view.

3. Compound Triggers

Combine multiple DML events (INSERT, UPDATE, DELETE) into **a single trigger block**.

Useful to:

- Handle complex business rules,
- Prevent **mutating table errors**,
- Use **bulk processing** for better performance.

4. DDL Triggers (Data Definition Language)

These are fired when DDL statements are executed:

- CREATE, ALTER, DROP
- TRUNCATE, RENAME
- GRANT, REVOKE

Mostly used for **auditing or enforcing database policies**.

5. System Triggers

Triggered based on **system-level events**, such as:

- LOGON / LOGOFF
- STARTUP / SHUTDOWN

Often used for **monitoring, security checks, or logging user sessions**.

Problem Statement – Create a Trigger

Question:

You need to create a trigger in an Oracle database that automatically increases the salary of employees who were hired in 2022 by 22.5% of their base salary. This trigger should activate after an INSERT or UPDATE operation on the employees table to ensure that any new employee hired in 2022 receives the salary increase right away.



Solution: Salary Increment

Solution

To accomplish this, you will define a **DML Trigger** that will execute after a new employee is inserted or an existing employee's hiring date is updated. The trigger will check if the hiring date falls within the year 2022 and, if so, apply the 22.5% increase to the employee's salary.

```
CREATE TABLE employees_test (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50) NOT NULL,  
    hire_date DATE NOT NULL,  
    salary NUMBER(10, 2)  
);
```

employees_test

Insert record 1 employees_test c

```
INSERT INTO employees (employee_id, first_name, hire_date, salary)  
VALUES (1, 'John', TO_DATE('2022-06-15', 'YYYY-MM-DD'), 50000);
```

- Drop Procedure
- Create new procedure

```
DROP TRIGGER trg_increase_salary;  
  
CREATE OR REPLACE TRIGGER trg_increase_salary  
BEFORE INSERT OR UPDATE ON employees_test  
FOR EACH ROW  
BEGIN  
    BEGIN  
        IF EXTRACT(YEAR FROM :NEW.hire_date) = 2022 THEN  
            :NEW.salary := :NEW.salary * 1.225;  
        END IF;  
    EXCEPTION  
        WHEN OTHERS THEN  
            DBMS_OUTPUT.PUT_LINE('Error in trg_increase_salary: ' || SQLERRM);  
            RAISE;  
    END;  
END;  
/
```

Solution: Salary Increment & Update & Test

Insert record 1 employees_test

```
INSERT INTO employees_test (employee_id, first_name, hire_date, salary)
VALUES (3, 'Kaka', TO_DATE('2022-03-10', 'YYYY-MM-DD'), 7000);
```

-- Check result

```
SELECT * FROM employees_test WHERE employee_id = 3;
```

```
UPDATE employees_test
SET hire_date = TO_DATE('2022-09-01', 'YYYY-MM-DD'),
    salary = 40000
WHERE employee_id = 3;
```

-- Check result

```
SELECT * FROM employees_test WHERE employee_id = 3;
```

The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL script:

```
25 SELECT * FROM employees_test WHERE employee_id = 3;
26
27
28
29 UPDATE employees_test
30 SET hire_date = TO_DATE('2022-09-01', 'YYYY-MM-DD'),
31     salary = 40000
32 WHERE employee_id = 3;
33
34 -- Check result
35 SELECT * FROM employees_test WHERE employee_id = 3;
```

Below the script, the 'Query Result' tab shows the output of the final SELECT statement. The status bar indicates 'All Rows Fetched: 3 in 0.001 seconds'. The resulting table is as follows:

EMPLOYEE_ID	FIRST_NAME	HIRE_DATE	SALARY
1	John	20-JUL-22	61250
2	Alice	10-MAR-22	49000
3	Kaka	01-SEP-22	49000

Output

Exercise: Enhancing Accountability and Insights Through Employee Salary Change Tracking

Exercise: Enhancing Accountability and Insights Through Employee Salary Change Tracking

Problem Statement

Context:

In a fast-paced business environment, managing employee compensation is critical for retaining talent and fostering employee satisfaction. Salary adjustments—whether due to performance evaluations, promotions, or market shifts—occur frequently and must be tracked transparently. A clear mechanism for monitoring these changes helps HR ensure accountability and make data-driven compensation decisions.

Challenge:

How can the organization implement an effective strategy to monitor and analyze salary changes recorded in the EMPLOYEES table, ensuring transparency, accountability, and strategic decision-making?

Questions to Explore:

1. How can we systematically log each salary change—including the old salary, new salary, and the difference—to create a reliable historical record of compensation adjustments?
2. What potential effects might salary changes have on employee performance, satisfaction, and organizational equity?
3. How can HR utilize the recorded salary change data to identify trends, assess fairness, and optimize compensation policies?
4. How can we implement this logging mechanism (e.g., using a trigger like `display_salary_changes`) in a way that minimizes performance overhead, especially during bulk data operations?

Objective

Design and implement a robust mechanism (such as a trigger) that automatically logs all salary changes made to the EMPLOYEES table. This mechanism should support real-time monitoring, empower HR with actionable insights, and promote fairness and transparency across the organization.



Exercise - Problem statement that relates to the trigger functionality and its implications in a business context:

Question: Tracking Gold Change

Question: Tracking Gold Stock Changes in Ngali Mining Ltd: Logging Updates for Mines Except CEO-Owned Operations

In a mining company like **Ngali Mining Ltd**, especially in a gold mining operation, it is essential to track changes related to the management and allocation of resources like gold production and compensation adjustments. The company aims to ensure transparency in its operations, monitor resources effectively, and maintain fair practices.

Given the MINES table in **Ngali Mining Ltd's Gold Mining System**, create a **conditional trigger** named `log_gold_changes` that fires **before** any DELETE, INSERT, or UPDATE operation on the table. The trigger should only log gold-related changes for mines that do not belong to the **CEO** or top executives. For other mines, it should log the **mine name**, **old gold stock**, **new gold stock**, and the **difference in gold**.

Specifications:

- **Trigger Activation:**
The trigger should fire only when the mine's owner is **not** the 'CEO' (or any top executives).
- **Output Format:**
The output should log the mine's name, old gold stock, new gold stock, and the difference in gold.

Objective:

The goal of this trigger is to ensure that **Ngali Mining Ltd** can easily track and review changes in gold stock levels for all mines except those owned by the CEO or top executives. This mechanism will help the company maintain a transparent record of resource changes, aiding in operational decision-making and fair distribution of gold production.



Difference Between Cursor and Trigger in PL/SQL

Cursor	Trigger
Processes rows one by one.	Executes automatically on events.
Can be defined inside a trigger.	Cannot be created within a cursor.
Created during SQL execution.	Stored in the database, triggered by events.
Activated by SQL statements.	Triggered by DDL/DML events.

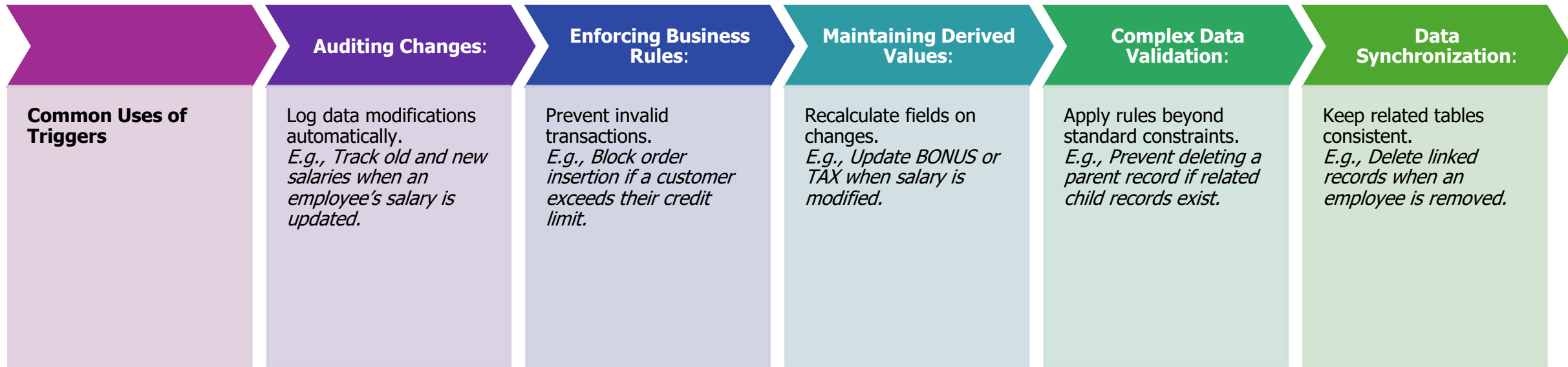
4. Conclusion

- **Cursors:** Best for controlled, row-by-row data processing.
- **Triggers:** Ideal for automating responses to database events to maintain data consistency.



Application of Triggers

Triggers are used to automatically execute PL/SQL code in response to specific database events like INSERT, UPDATE, DELETE, or DDL operations (CREATE, ALTER, DROP). They help maintain data integrity, enforce business logic, audit activity, and automate data processes without user intervention.



Be Cautious: What to Know Before Using Triggers

Triggers are used to automatically execute PL/SQL code in response to specific database events like INSERT, UPDATE, DELETE, or DDL operations (CREATE, ALTER, DROP). They help maintain data integrity, enforce business logic, audit activity, and automate data processes without user intervention.

Triggers are special procedures that automatically execute in response to specific events on a table or view. They are commonly used to **enforce business rules, ensure data integrity, audit data changes, and automate workflows** without manual intervention.

However, while triggers are powerful, they should be used with care:

Avoid complex logic that can affect performance.

Be mindful of **mutating table errors**.

Ensure that triggers **do not create unintended recursive effects**.

Always **test thoroughly** to confirm expected behavior.

You will never change your life until you change something you do daily. The secret of your success is found in your daily routine.

– John C. Maxwell

Keep Learning!